AP & DE CAPSTONE PROJECT · STAGE 1 EVALUATION

# SkillBridge

A student collaboration platform for teammate discovery, project teams and peer-to-peer skill exchange

## Ideation, Architecture and Database Design Document

| | |
|---|---|
| Course | Advanced Programming (AP) and Data Engineering (DE) · Academic Year 2026-27 |
| Evaluation | Stage 1: Ideation, Architecture and Database Design · 13 to 24 October 2026 |
| Team member 1 | Name: _____ Roll no.: _____ |
| Team member 2 | Name: _____ Roll no.: _____ |
| Faculty reviewer | _____ |
| Topic reference | Topic no. / title from the professor's list: _____ |
| GitHub repository | https://github.com/_____ |
| Document version | 1.0 · 18 October 2026 · Draft for faculty review |

Status of this document: this is a design document. Nothing described here is claimed as implemented yet; every feature, test and deployment step is a plan to be delivered in the build phase.

# Table of contents

**1. Introduction . . . 5**

1.1 Purpose of this document . . . 5

1.2 Stage 1 deliverables checklist . . . 5

1.3 Scope of the product . . . 5

1.4 Definitions and abbreviations . . . 5

**2. Problem statement and ideation . . . 6**

2.1 Background and problem . . . 6

2.2 Problems and proposed solutions . . . 6

2.3 Proposed solution overview . . . 6

2.4 Target users . . . 6

2.5 User journey . . . 6

2.6 Feature breakdown (MVP) . . . 7

2.7 Out of scope for the MVP . . . 7

2.8 Functional requirements and acceptance criteria . . . 7

2.9 Non-functional requirements . . . 8

2.10 Business rules . . . 8

**3. End goal definition . . . 9**

3.1 Target deliverable . . . 9

3.2 Definition of done (MVP acceptance checklist) . . . 9

3.3 Demonstration scenario for Stage 2 . . . 9

**4. System architecture and technology stack . . . 10**

4.1 Architectural style . . . 10

4.2 Responsibilities of each layer . . . 10

4.3 Technology choices and justification . . . 11

4.4 Key architectural decisions . . . 11

4.5 Deployment architecture (planned) . . . 12

**5. Database design . . . 13**

5.1 Design principles . . . 13

5.2 Data dictionary (tables) . . . 18

5.3 Relationships and cardinality . . . 18

5.4 Constraints . . . 19

5.5 Indexing strategy . . . 19

5.6 Normalization to 5NF . . . 20

5.7 Why some obvious columns are missing . . . 21

5.8 Transactions and concurrency . . . 22

5.9 Analytics view for the Data Engineering component . . . 22

5.10 Seed data plan . . . 22
**6. Data flow diagrams . . . 23**
6.1 Notation . . . 23
6.2 Level 0: context diagram . . . 23
6.3 Level 1: main processes and data stores . . . 24
6.4 Level 2: process 4.0 (applications and teams) . . . 25
**7. Low-level design . . . 27**
7.1 Request lifecycle . . . 27
7.2 Backend structure and conventions . . . 27
7.3 Module catalogue . . . 27
7.4 REST API . . . 28
7.5 State machines . . . 30
7.6 Key algorithms . . . 31
7.7 Error handling and logging . . . 32
7.8 Frontend low-level design . . . 32
**8. Security design . . . 34**
8.1 Threats and controls . . . 34
8.2 Permission matrix . . . 34
**9. Quality assurance and testing strategy . . . 35**
9.1 Test levels . . . 35
9.2 Test case matrix . . . 35
9.3 Continuous integration . . . 35
**10. Deployment and operations plan . . . 36**
10.1 Configuration (environment variables) . . . 36
10.2 Release checklist . . . 36
**11. Project plan, team and risks . . . 37**
11.1 Timeline . . . 37
11.2 Team and responsibilities (proposed) . . . 37
11.3 Risk register . . . 37
**12. GitHub repository and documentation plan . . . 39**
12.1 Repository conventions . . . 39
12.2 Initial repository contents (Stage 1) . . . 39
12.3 Stage 1 repository checklist . . . 39
**13. Mapping to the evaluation framework . . . 40**
13.1 Stage 1 (13 to 24 October) . . . 40
13.2 Stage 2 (26 November to 4 December), planned . . . 40
**14. Resource attribution and academic integrity . . . 40**

**Appendix A. Prisma schema (source of the ERD) . . . . . 41**

**Appendix B. Raw SQL added to the first migration . . . . . 44**

**Appendix C. Standard error codes . . . . . 44**

# 1. Introduction

## 1.1 Purpose of this document

This document is the complete Stage 1 submission for the SkillBridge capstone project. The evaluation framework for AP and DE asks each team to present a problem statement with ideation, the end goal, the technology architecture with justification, database diagrams and an initialised public GitHub repository. In addition, the faculty asked for an entity-relationship diagram that satisfies every normal form from 1NF to 5NF, a low-level design and a data flow diagram. All of these are included here so that the design can be reviewed and approved before implementation starts.

## 1.2 Stage 1 deliverables checklist

| Required deliverable | Where in this document | Status |
|---|---|---|
| Problem statement and ideation (domain problem, solution, feature breakdown) | Section 2 | Included |
| End goal definition (website, mobile app or prototype) | Section 3 | Included: deployed responsive website |
| Tech stack architecture with technical justification | Section 4 | Included |
| Database diagram: ER diagram for SQL (faculty: 1NF to 5NF) | Section 5 and Appendix A | Included, with normalization proof |
| Data flow diagram (faculty request) | Section 6 | Included: Level 0, 1 and 2 |
| Low-level design (faculty request) | Section 7 | Included |
| Initialised public GitHub repository with setup and initial documentation | Section 12 | Plan and checklist included; repository link goes on the cover page |
| Resource attribution (libraries, AI tools) as required by the handbook | Section 14 | Included |

## 1.3 Scope of the product

SkillBridge is a full-stack web application. The first release (MVP) contains seven modules: authentication, student profiles and skills, project discovery and management, applications and team formation, peer skill exchange, explainable teammate matching and in-app notifications. Team task boards, real-time chat, email notifications and university verification are explicitly out of scope for the MVP (see Section 2.7).

## 1.4 Definitions and abbreviations

| Term | Meaning |
|---|---|
| MVP | Minimum viable product: the smallest complete version that delivers the core value. |
| ERD | Entity-relationship diagram: tables, keys and relationships of the database. |
| DFD | Data flow diagram: how data moves between external entities, processes and data stores. |
| LLD | Low-level design: modules, interfaces, algorithms and rules at implementation level. |
| FD, MVD, JD | Functional, multivalued and join dependency: the dependencies used to prove normal forms. |
| 1NF to 5NF, BCNF | Normal forms. BCNF sits between 3NF and 4NF. |
| ORM | Object-relational mapper. This project uses Prisma. |
| REST | Architectural style for HTTP APIs using resources and standard methods. |
| CSRF | Cross-site request forgery: tricking a browser into sending an authenticated request. |
| CRUD | Create, read, update, delete. |

# 2. Problem statement and ideation

## 2.1 Background and problem

Engineering students are expected to build real projects, but most struggle to start. A student who knows some JavaScript and React may have a good product idea and still lack backend, database and deployment skills. Other students have exactly those skills and want a project to apply them to. Today these students find each other only by chance, through friends or crowded group chats, so good ideas stall and useful skills go unused.

**Problem statement:** students have no reliable, structured way to find teammates with complementary skills, to find projects that need their skills, or to trade knowledge with peers while building something real. As a result projects remain unfinished and students cannot show practical experience.

## 2.2 Problems and proposed solutions

| Problem | SkillBridge solution |
|---|---|
| Students struggle to find suitable teammates | Skill-based discovery and explainable teammate matching |
| Project ideas stall because the team lacks skills | Project listings with required skills, levels and open positions |
| Students do not know where to get practical experience | Project discovery with search, filters and applications |
| Students want to learn but mentoring is not always affordable | Peer-to-peer skill exchange (offer one skill, request another) |
| Students cannot show what they actually contributed | Profiles with skills, team memberships and project links |

## 2.3 Proposed solution overview

A student creates a profile with the skills they know (with a proficiency level), the skills they want to learn and their interests. They can publish a project with required skills and a team size, or browse projects and apply to join. The project owner reviews applications and forms a team; the platform guarantees the team never exceeds its capacity. Students can also send skill-exchange requests to each other. For project owners, the matching engine ranks students by fit and explains each ranking in plain language.

**Honest positioning.** Project directories and profile sites already exist, so SkillBridge does not claim uniqueness for those. Its differentiation is (1) a transparent, rule-based matching score that shows its reasons and (2) a structured skill-exchange flow that sits next to project teams. The matching score is a ranking aid, not a prediction of project success.

## 2.4 Target users

| Persona | Example | Main goals | Main features used |
|---|---|---|---|
| Project beginner | Student learning React and Node.js | Find a beginner-friendly project, learn by doing, gain resume value | Discover projects, apply, exchange skills |
| Project founder | Student with a startup or hackathon idea | Assemble a reliable team with complementary skills | Create project, define required skills, review applications, matching |
| Skill sharer | Student who knows Figma and wants backend skills | Help others and learn in return | Offer and request skills, manage exchanges |

One person can play all three roles, so there is a single student account type. 'Project owner' and 'team member' are roles inside a project, not account types.

## 2.5 User journey

1. **Register and log in** with name, email and password. The server validates input, hashes the password and starts a session.
2. **Build a profile**: bio, college, links, skills with levels, learning goals, interests.
3. **Discover projects** using search and filters (skill, category, difficulty, status).

4. **Apply** with a short introduction. The owner reviews and accepts or rejects.
5. **Form a team**: on acceptance the server checks capacity and creates the membership in one transaction.
6. **Find teammates**: the owner opens the match list for the project and reads why each student fits.
7. **Exchange skills**: students send, accept and complete skill-exchange requests.

## 2.6 Feature breakdown (MVP)

| ID | Module | Feature | Priority |
|---|---|---|---|
| F-01 | Authentication | Register, login, logout, current-user endpoint, protected pages, session expiry | Must |
| F-02 | Profiles | View and edit profile; manage skills with levels, learning goals, interests and links | Must |
| F-03 | Skill catalogue | Canonical skill list with categories and aliases (so 'NodeJS' and 'Node.js' are one skill) | Must |
| F-04 | Projects | Create, edit, publish, archive; required skills with minimum level; team size; links | Must |
| F-05 | Discovery | Search by keyword; filter by skill, category, difficulty, status; sort; pagination | Must |
| F-06 | Applications | Apply, withdraw, view own applications; owner reviews and accepts or rejects | Must |
| F-07 | Teams | Membership created on acceptance; capacity never exceeded under concurrency | Must |
| F-08 | Skill exchange | Send, accept, reject, cancel and complete exchange requests | Must |
| F-09 | Matching | Ranked candidates for a project with score breakdown and reasons | Must |
| F-10 | Notifications | In-app notification list, unread count, mark read | Should |
| F-11 | Dashboard | Recommended projects, pending applications, incoming exchanges, recent notifications | Should |
| F-12 | Analytics view | SQL view of skill demand versus supply (supports the DE component) | Should |

## 2.7 Out of scope for the MVP

- Team workspace with tasks, milestones and GitHub integration (future enhancement).
- Real-time chat, video calls and scheduling inside skill exchanges.
- Email notifications and password recovery by email.
- University communities, verified student accounts and contribution verification.
- Machine-learning recommendations. The MVP uses a deterministic, explainable scoring rule.

## 2.8 Functional requirements and acceptance criteria

| ID | Requirement (user story) | Acceptance criterion |
|---|---|---|
| FR-01 | As a visitor I can register with name, email and password. | Duplicate email returns 409; password stored only as a hash; response never contains the hash. |
| FR-02 | As a student I can log in and out. | Wrong credentials return a generic 401; logout invalidates the server-side session. |
| FR-03 | As a student I can edit my profile and skills. | Skills must exist in the catalogue; each skill has a level; changes visible on my public profile. |
| FR-04 | As an owner I can create and edit my projects. | Only the owner can edit; team size between 2 and 10; status changes follow the allowed transitions. |
| FR-05 | As a student I can search and filter projects. | Empty result returns 200 with an empty list; page size is capped on the server. |
| FR-06 | As a student I can apply to an open project. | Cannot apply to own project, to a non-open project, or twice. |
| FR-07 | As an owner I can accept or reject applications. | Only the owner can decide; an already-decided application returns 409. |
| FR-08 | As an owner I never exceed the team size. | Two concurrent accepts for the last seat: exactly one succeeds. |

| ID | Requirement (user story) | Acceptance criterion |
|---|---|---|
| FR-09 | As a student I can withdraw a pending application. | Only the applicant can withdraw; only while PENDING. |
| FR-10 | As a student I can request a skill exchange. | Not with myself; offered skill is on my profile; requested skill is on the recipient's profile; no duplicate active request. |
| FR-11 | As a recipient I can accept or reject an exchange; participants can complete it. | Only allowed transitions succeed; only participants can view the exchange. |
| FR-12 | As an owner I can see ranked candidates for my project. | Owner and current members excluded; every result has a score and reasons. |
| FR-13 | As a student I see notifications for events that concern me. | A notification exists only if the business operation succeeded; I can read only my own. |

## 2.9 Non-functional requirements

| Area | Requirement (target to be verified during testing) |
|---|---|
| Security | Hashed passwords; HTTP-only session cookie; server-side authorisation on every protected operation; validated input; rate-limited login. |
| Data integrity | Foreign keys, unique and CHECK constraints in the database; multi-step changes in transactions. |
| Performance | List endpoints paginated with a server-side cap; indexes on filter and join columns; target under 500 ms for the project list on the seeded demo data. |
| Reliability | Central error handling; a health endpoint; migrations tracked in version control. |
| Usability | Loading, empty and error states on every page; responsive from phone to desktop. |
| Accessibility | Semantic HTML, keyboard-reachable controls, visible focus, sufficient colour contrast. |
| Maintainability | Module-per-feature backend; consistent file layout; linting and formatting; automated tests for business rules. |
| Observability | Structured JSON logs with request id; business events logged; secrets and passwords redacted. |
| Portability | Configuration only through environment variables; same code runs locally and in the cloud. |

## 2.10 Business rules

| ID | Rule |
|---|---|
| BR-01 | Email is unique and compared in lower case. |
| BR-02 | A user has at most one level per skill and at most one link per link type. |
| BR-03 | A project has one owner. The owner occupies one seat; seats taken = 1 + number of members. |
| BR-04 | Team size (including the owner) is between 2 and 10. |
| BR-05 | Applications are accepted only while the project is OPEN. |
| BR-06 | A user cannot apply to own project and has one application per project. |
| BR-07 | Only the owner decides applications; only the applicant can withdraw. |
| BR-08 | Accepting an application creates the membership and updates the application in one transaction, never exceeding capacity. |
| BR-09 | A rejected application is final. A withdrawn application can be reopened by the applicant while the project is OPEN. |
| BR-10 | A user cannot exchange skills with themselves. |
| BR-11 | The offered skill must be on the sender's profile; the requested skill must be on the recipient's profile. |
| BR-12 | Only one active (PENDING or ACCEPTED) exchange may exist for the same sender, recipient and skill pair. |
| BR-13 | Matching excludes the owner and current team members; absence of a skill on a profile is not proof the person lacks it. |
| BR-14 | Team size, 'project is full' and unread counts are derived on read, never stored. |

# 3. End goal definition

## 3.1 Target deliverable

**Final deliverable: a responsive, deployed website** (React single-page application with a Node.js and Express REST API backed by PostgreSQL), reachable at a public HTTPS address, with a public GitHub repository containing source, documentation and tests. The evaluation framework reserves one mark for live cloud deployment, so a localhost-only demo is not an acceptable end state.

## 3.2 Definition of done (MVP acceptance checklist)

| # | Criterion | Verified by |
|---|---|---|
| 1 | Users can register, log in, log out and reach protected pages only when authenticated. | Integration tests, live demo |
| 2 | Users can edit profiles, manage skills with levels and set learning goals. | Integration tests, live demo |
| 3 | Users can create, edit, discover, search and filter projects. | Integration tests, live demo |
| 4 | Students can apply, withdraw and see application status. | Integration tests, live demo |
| 5 | Owners can review applications and form teams without exceeding capacity. | Concurrency test, live demo |
| 6 | Students can send, manage and complete valid skill-exchange requests. | Integration tests, live demo |
| 7 | Matching returns eligible candidates with understandable reasons. | Unit tests, live demo |
| 8 | Authorisation, validation, database constraints and error handling are implemented. | Negative tests, code defence |
| 9 | Important business rules and API workflows have automated tests that run in CI. | CI run on GitHub |
| 10 | The application is deployed; README, setup guide and API documentation exist. | Public URL, repository |

## 3.3 Demonstration scenario for Stage 2

1. Two student accounts register; one builds a profile with React and JavaScript, the other with Node.js and PostgreSQL.
2. The second student creates a project requiring React, Node.js and PostgreSQL with a team size of three.
3. The first student searches, finds the project and applies; the owner receives a notification.
4. The owner opens the match list, sees the score breakdown, then accepts the application. Membership appears and the project shows 'seats left'.
5. A third account applies for the last seat while the demo script fires two accept requests at once; exactly one succeeds.
6. The students exchange skills (React for PostgreSQL) and mark the exchange completed.
7. Show logs, the database constraints rejecting invalid data, and the test suite passing in CI.

# 4. System architecture and technology stack

## 4.1 Architectural style

SkillBridge is a **three-tier, layered modular monolith**: a React single-page application (presentation), a Node.js and Express REST API (application) and a PostgreSQL database (data). The backend is one deployable unit, internally divided into feature modules with the same internal layers. For a two-person, five-week build this gives clear separation of concerns, simple deployment and easy debugging, without the operational cost of microservices, queues or Kubernetes. Because all business rules live behind the API, a mobile app or another client could reuse the same backend later.

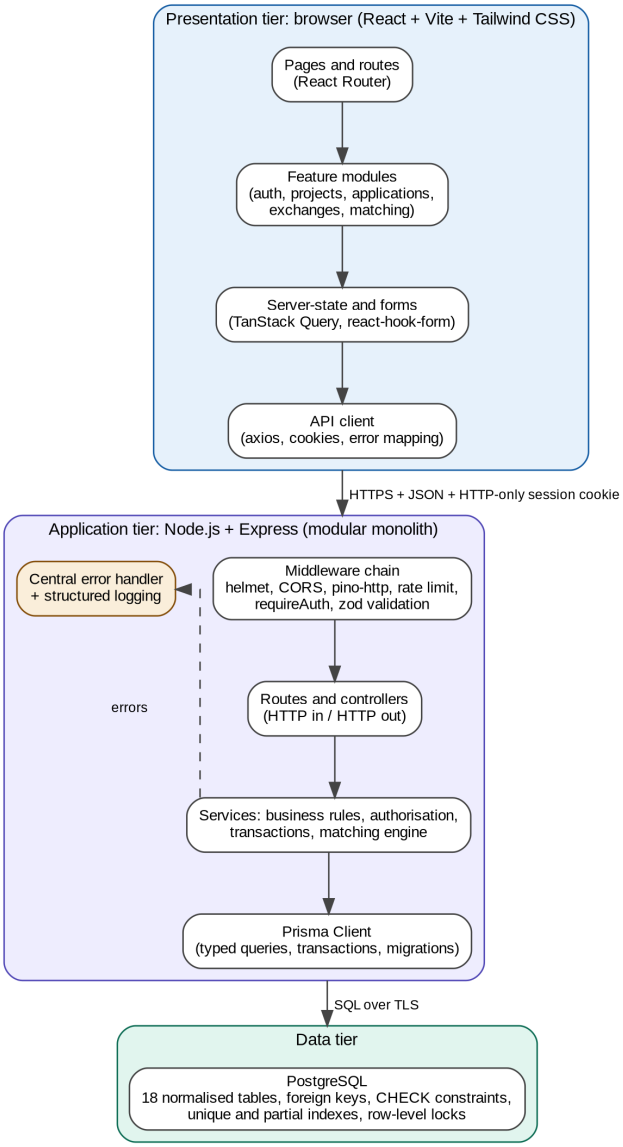



*Figure 4.1: Logical architecture. The browser never talks to the database; every access passes through the API.*

## 4.2 Responsibilities of each layer

| Layer | Responsibility | Must not do |
|---|---|---|
| React client | Screens, forms, routing, loading/empty/error states, calling the API | Enforce security or business rules (hiding a button is not security) |
| Middleware | Security headers, CORS, request logging, rate limiting, session check, input validation | Contain business logic |
| Routes and controllers | Map URLs to service calls; translate HTTP in and out | Talk to the database directly |

| Layer | Responsibility | Must not do |
|---|---|---|
| Services | Business rules, authorisation, transactions, matching algorithm | Know about HTTP objects |
| Prisma | Typed queries, relations, transactions, migrations | Hold business rules |
| PostgreSQL | Durable storage; constraints, indexes and locks as the last line of defence | Be bypassed by the frontend |

## 4.3 Technology choices and justification

| Layer | Choice | Justification | Alternatives considered |
|---|---|---|---|
| Frontend framework | React 18 with Vite | Component model suits dashboards and forms; large ecosystem; fast dev server and build; the team already knows JavaScript | Angular (heavier, steeper learning curve); plain JavaScript (no structure for a multi-page app) |
| Styling | Tailwind CSS | Consistent responsive layouts without writing a large custom stylesheet | Bootstrap, plain CSS |
| Backend runtime | Node.js | Same language as the frontend; non-blocking I/O fits an API-heavy app | Java Spring (more setup), Python Django |
| Web framework | Express.js | Small and explicit; every middleware step is visible and explainable in a code defence | NestJS (more structure, more concepts to learn), Fastify |
| Database | PostgreSQL | The data is strongly relational (many-to-many tables, constraints, transactions, row locks, partial indexes) | MongoDB (weak fit for many-to-many integrity and multi-document transactions), MySQL |
| ORM | Prisma | Typed schema as single source of truth, migrations tracked in Git, clean transaction API, can generate an ERD | Sequelize, TypeORM, raw SQL (more boilerplate) |
| Validation | zod | One schema validates body, query and params and produces structured field errors | Joi, express-validator |
| Authentication | bcrypt + server-side sessions in PostgreSQL | Sessions can be revoked immediately; only a token hash is stored; HTTP-only cookie is not readable by scripts | JWT in local storage (harder to revoke, exposed to XSS) |
| Logging | pino and pino-http | Fast structured JSON logs with request ids and redaction | winston, console.log |
| Testing | Vitest, Supertest, Playwright (optional) | Unit, API-integration and browser tests; Supertest runs against the real Express app and a test database | Jest, Cypress |
| Data fetching | TanStack Query, axios | Caching, loading and error states for server data | Hand-written fetch and state |
| Forms | react-hook-form with zod resolver | Same validation rules on client and server | Uncontrolled forms by hand |
| Security middleware | helmet, cors, express-rate-limit | Standard hardening for headers, cross-origin rules and brute-force protection | Custom code |
| Version control and CI | Git, GitHub, GitHub Actions | Required public repository; CI runs lint and tests on every push | GitLab |

## 4.4 Key architectural decisions

| Decision | Rationale | Trade-off accepted |
|---|---|---|
| Modular monolith | One deployable unit; modules share the database but have separate files and rules | Cannot scale modules independently (not needed at this size) |
| Server-side sessions | Instant revocation; no token content to leak | One database lookup per authenticated request |
| REST + JSON | Simple, cacheable, easy to test and document | Possible over-fetching; mitigated with explicit field selection |

| Decision | Rationale | Trade-off accepted |
|---|---|---|
| Rule-based matching | Explainable, testable, no training data required | Less adaptive than machine learning; weights are design choices, not validated values |
| Derived data never stored | Avoids update anomalies (see Section 5.7) | Counts are computed on read; indexed so cost is small |
| Rules also in the database | CHECK constraints, unique and partial indexes protect integrity even if application code has a bug | Some rules live in raw SQL migrations because Prisma cannot express them |
| Row lock for team capacity | Correct under concurrent accepts | Brief serialisation of decisions on one project (acceptable) |

## 4.5 Deployment architecture (planned)

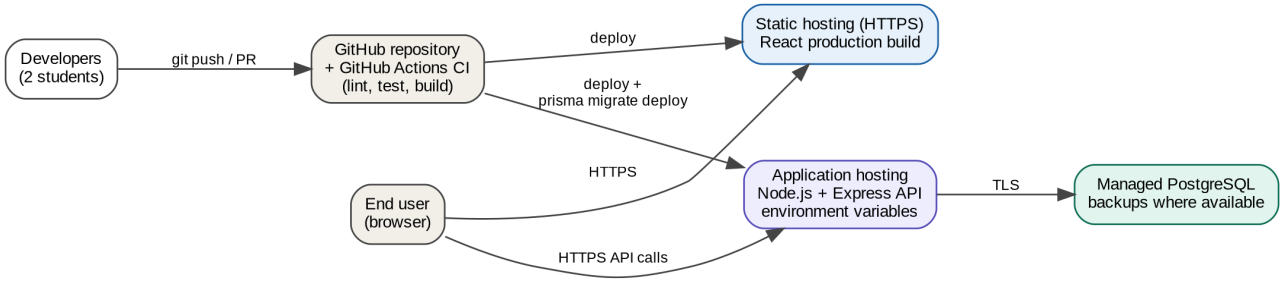



*Figure 4.2: Planned deployment. Concrete providers are chosen at deployment time because free-tier terms change.*

The frontend is served as static files over HTTPS, the API runs as a Node.js service with configuration in environment variables, and PostgreSQL is a managed instance. Migrations run with `prisma migrate deploy` during release. **Cookie caveat:** if the frontend and API live on different sites, the session cookie needs `SameSite=None; Secure` plus an Origin check and CSRF token; if both are under one parent domain, `SameSite=Lax` is enough. This is decided before the first deployment.

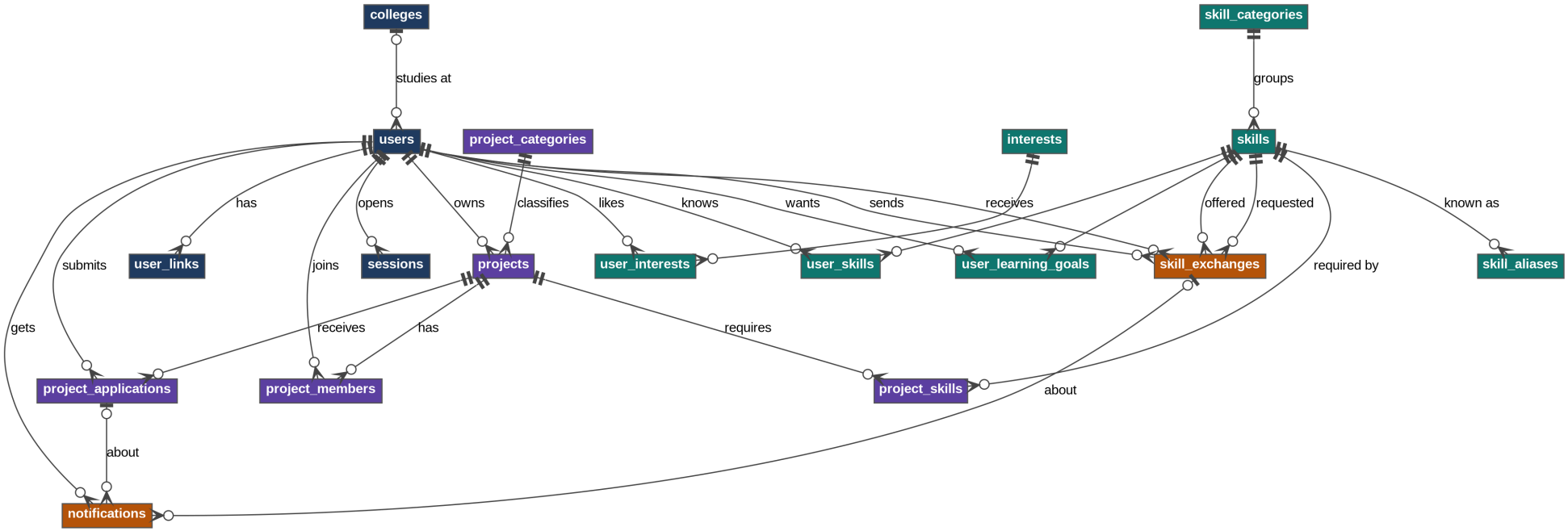
# 5. Database design

## 5.1 Design principles

- One fact in one place: every non-key column describes the key, the whole key and nothing but the key.
- Skills, interests, links and goals are rows in their own tables, never comma-separated strings.
- A canonical skill catalogue with aliases prevents 'Node.js' and 'NodeJS' becoming two skills.
- An application (a request) and a membership (a fact) are different things, so they are different tables.
- Derived values (team size, 'full', unread count, match score, notification text) are computed, not stored.
- The database enforces integrity with foreign keys, unique constraints, CHECK constraints and a partial unique index.

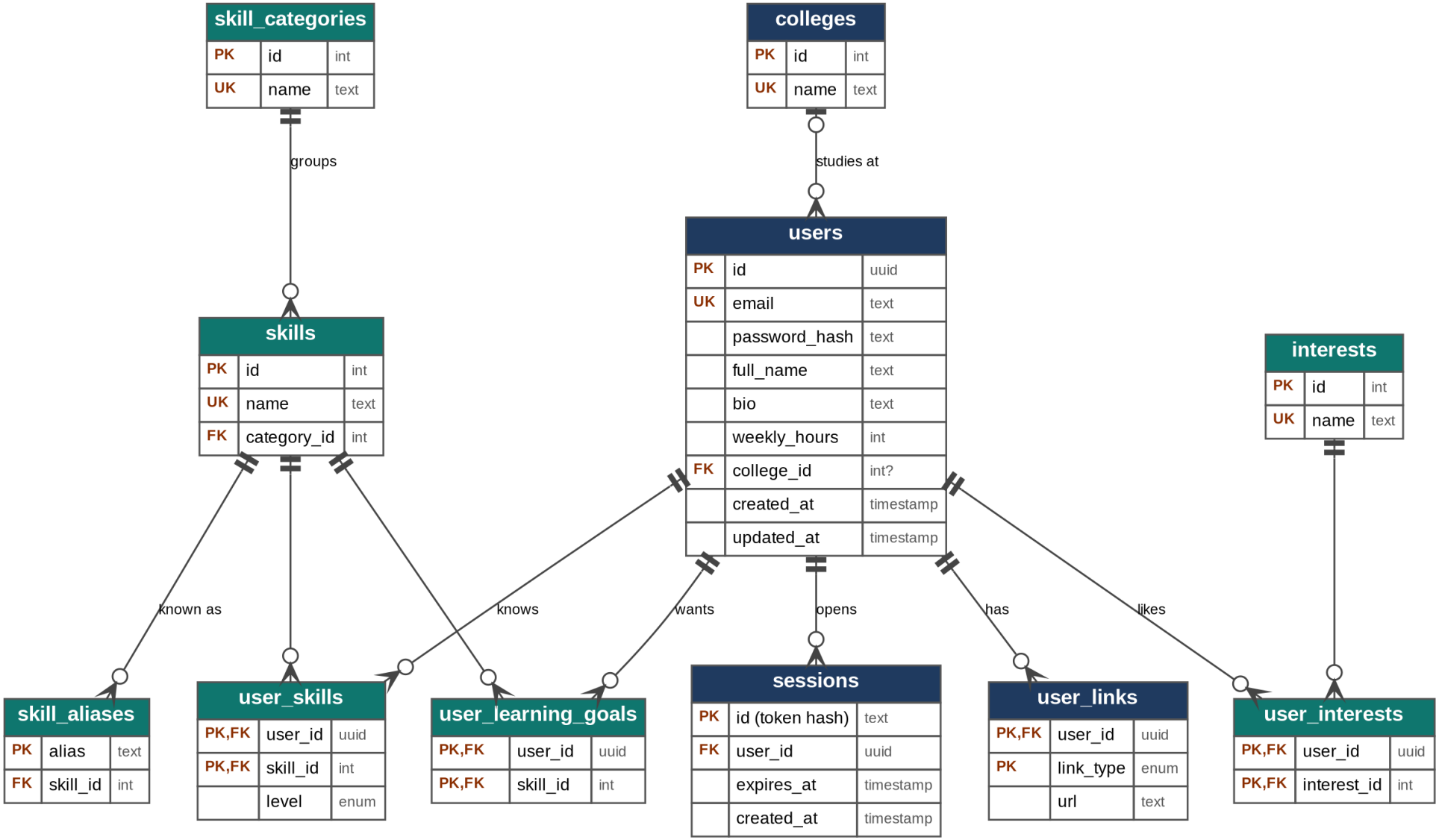
The ERD on the next page uses crow's-foot notation: a double bar at the parent end means exactly one, a bar with a circle means zero or one, and a crow's foot with a circle means zero or many. Header colours group the tables: blue = identity, green = skills and interests, purple = projects and teams, orange = exchanges and notifications. PK = primary key, FK = foreign key, UK = unique key, '?' = nullable column.

**Figure 5.1: ER overview of SkillBridge (18 tables, normalised to 5NF). Table names only; columns follow in Figures 5.2 to 5.4.**



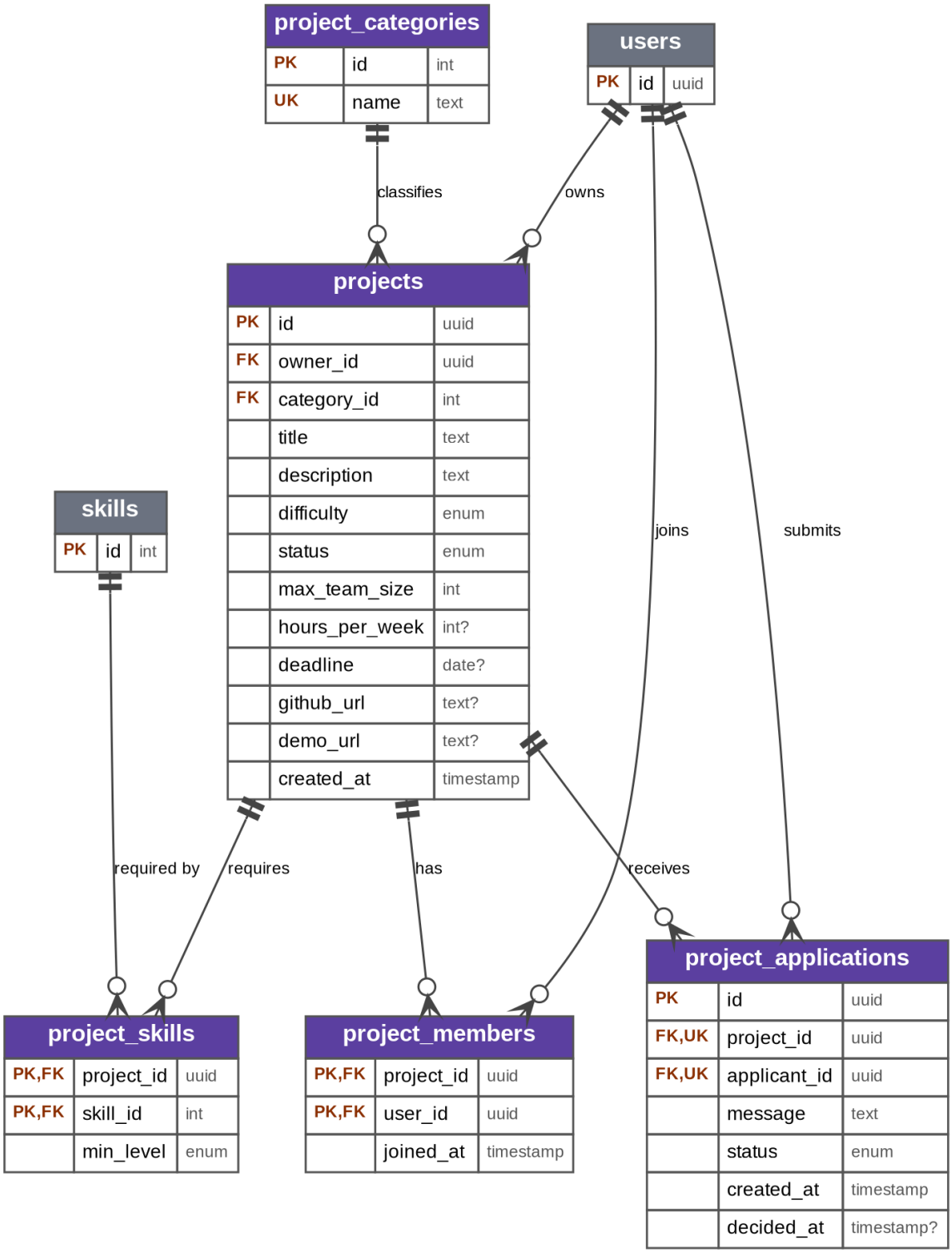

*Overview of all tables and relationships. Header colours: blue = identity, green = skills and interests, purple = projects and teams, orange = exchanges and notifications.*

### Figure 5.2: ERD detail, identity and skills domain (11 tables)



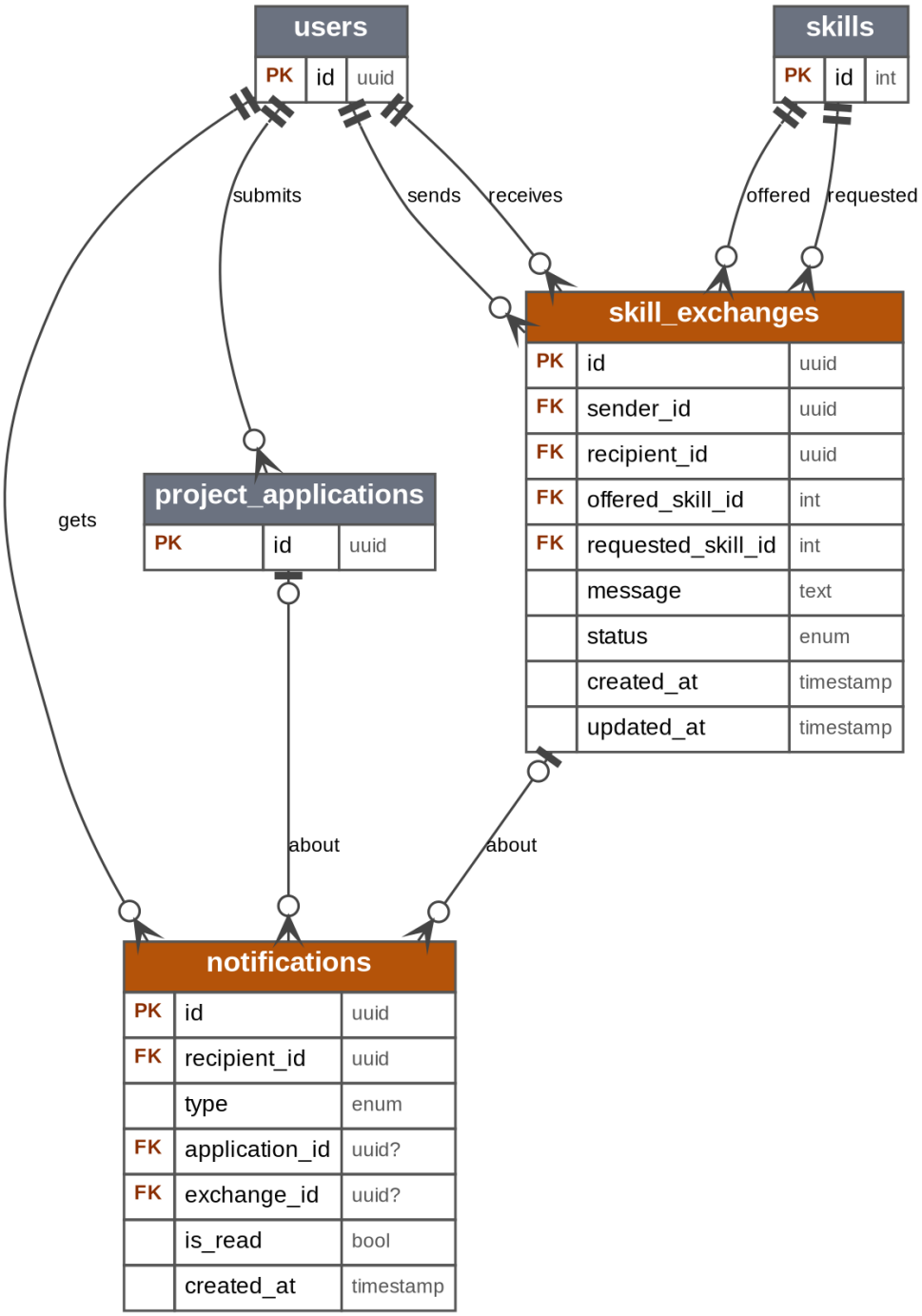

*Columns with key markers. PK = primary key, FK = foreign key, UK = unique key, '?' = nullable.*

### Figure 5.3: ERD detail, projects and teams domain




*Grey tables (users, skills) are shown with their key only to show the link to the identity and skills domain.*

### Figure 5.4: ERD detail, skill exchanges and notifications




*Grey tables are shown with their key only. A notification refers to exactly one application or one exchange (CHECK constraint).*

## 5.2 Data dictionary (tables)

| Table | Purpose | Primary key | Other keys and notes |
|---|---|---|---|
| colleges | Lookup of colleges | id | UK name |
| users | Account credentials and profile text | id | UK email (stored lower case); FK college_id (nullable) |
| user_links | GitHub, LinkedIn, portfolio links | (user_id, link_type) | One link per type per user |
| sessions | Server-side login sessions | id (SHA-256 of cookie token) | FK user_id; expires_at indexed |
| skill_categories | Groups of skills (Frontend, Backend, Design, ...) | id | UK name |
| skills | Canonical skill catalogue | id | UK name; FK category_id |
| skill_aliases | Alternate spellings mapped to a skill | alias (lower case) | FK skill_id |
| user_skills | Skills a user knows, with level | (user_id, skill_id) | level enum; index on skill_id |
| user_learning_goals | Skills a user wants to learn | (user_id, skill_id) | Index on skill_id |
| interests | Lookup of interests (SaaS, AI tools, ...) | id | UK name |
| user_interests | Interests of a user | (user_id, interest_id) | |
| project_categories | Lookup of project categories | id | UK name |
| projects | Project description, owner, status, capacity | id | FK owner_id, category_id; CHECK max_team_size 2..10; index (status, category_id) |
| project_skills | Skills a project requires, with minimum level | (project_id, skill_id) | min_level enum; index on skill_id |
| project_applications | Requests to join a project | id | UK (project_id, applicant_id); index (project_id, status) |
| project_members | Actual team members (excluding the owner) | (project_id, user_id) | joined_at; index on user_id |
| skill_exchanges | Skill-exchange requests between two users | id | FKs to sender, recipient, offered and requested skill; CHECK sender <> recipient; partial unique index on active requests |
| notifications | In-app alerts | id | FK recipient_id; exactly one of application_id or exchange_id (CHECK) |

## 5.3 Relationships and cardinality

| Relationship | Cardinality | Meaning |
|---|---|---|
| colleges - users | 0..1 to many | A user may belong to one college; a college has many users |
| users - user_links / sessions / user_interests | 1 to many | A user has many links, sessions and interests |
| users - skills (via user_skills) | many to many | A user knows many skills; a skill is known by many users |
| users - skills (via user_learning_goals) | many to many | A user wants to learn many skills; a skill is wanted by many users |
| skill_categories - skills | 1 to many | A skill has exactly one category |
| skills - skill_aliases | 1 to many | A skill has many spellings |
| users - projects (owner) | 1 to many | A user owns many projects; a project has exactly one owner |
| project_categories - projects | 1 to many | A project has exactly one category |

| Relationship | Cardinality | Meaning |
|---|---|---|
| projects - skills (via project_skills) | many to many | A project requires many skills; a skill is required by many projects |
| projects - users (via project_applications) | many to many with attributes | A user applies to many projects; a project receives many applications; one per pair |
| projects - users (via project_members) | many to many | A project has many members; a user joins many projects |
| users - users (via skill_exchanges) | many to many with attributes | Sender and recipient, each with an offered and a requested skill |
| users - notifications | 1 to many | A user receives many notifications |
| applications / exchanges - notifications | 0..1 to many | A notification refers to exactly one application or one exchange |

## 5.4 Constraints

| Constraint | Where | Protects |
|---|---|---|
| Primary keys, composite primary keys | All tables | Row identity; no duplicate skill for a user, no duplicate required skill, no duplicate member |
| Foreign keys with ON DELETE rules | All relationships | No orphan rows; cascade for dependent rows (links, sessions, user skills), restrict for shared references |
| UNIQUE users.email | users | No duplicate accounts (BR-01) |
| UNIQUE (project_id, applicant_id) | project_applications | One application per user per project (BR-06) |
| CHECK max_team_size BETWEEN 2 AND 10 | projects | Valid capacity (BR-04) |
| CHECK sender_id <> recipient_id | skill_exchanges | No self-exchange (BR-10) |
| Partial UNIQUE index on (sender, recipient, offered, requested) WHERE status IN (PENDING, ACCEPTED) | skill_exchanges | No duplicate active exchange (BR-12) |
| CHECK num_nonnulls(application_id, exchange_id) = 1 | notifications | Each notification points to exactly one source |
| Enumerations for level, status, type | Prisma enums (PostgreSQL enum types) | Only legal values stored |

## 5.5 Indexing strategy

| Index | Supports |
|---|---|
| users(email) unique | Login lookup and duplicate detection |
| sessions(user_id), sessions(expires_at) | Logout-everywhere and cleanup of expired sessions |
| projects(status, category_id) | Project list filters |
| projects(owner_id) | 'My projects' page and ownership checks |
| project_skills(skill_id) | Filter projects by skill; matching pre-filter |
| user_skills(skill_id), user_learning_goals(skill_id) | Matching: find students who have or want a required skill |
| project_applications(project_id, status), (applicant_id) | Owner's review queue and 'my applications' |
| project_members(user_id) | 'Projects I joined' |
| skill_exchanges(recipient_id, status), (sender_id, status) | Incoming and outgoing exchange lists |
| notifications(recipient_id, is_read, created_at) | Notification list and unread count |

## 5.6 Normalization to 5NF

A normal form is proved against declared dependencies, so the assumptions come first.

| # | Assumption (dependency declared) |
|---|---|
| A1 | Email identifies a user: email determines user_id. |
| A2 | A skill belongs to exactly one category: skill_id determines category_id. |
| A3 | A project has exactly one owner and one category: project_id determines owner_id and category_id. |
| A4 | A user has one level per skill: (user_id, skill_id) determines level. |
| A5 | A user has at most one link of each type: (user_id, link_type) determines url. |
| A6 | The same two students may have several exchanges over time, so (sender, recipient, offered, requested) is not a key; only the surrogate id is. |

### Functional dependencies and highest normal form per table

| Table | Candidate keys | Non-trivial FDs (every determinant is a candidate key) | Result |
|---|---|---|---|
| users | id; email | id -> all columns; email -> all columns | BCNF |
| colleges, skill_categories, project_categories, interests | id; name | id -> name; name -> id | BCNF |
| skills | id; name | id -> name, category_id; name -> id, category_id | BCNF |
| skill_aliases | alias | alias -> skill_id | BCNF |
| user_links | (user_id, link_type) | (user_id, link_type) -> url | BCNF |
| sessions | id | id -> user_id, expires_at, created_at | BCNF |
| user_skills | (user_id, skill_id) | (user_id, skill_id) -> level | BCNF |
| user_learning_goals, user_interests | all columns | only trivial FDs | BCNF and 5NF |
| projects | id | id -> all columns | BCNF |
| project_skills | (project_id, skill_id) | (project_id, skill_id) -> min_level | BCNF |
| project_applications | id; (project_id, applicant_id) | each key -> all columns | BCNF |
| project_members | (project_id, user_id) | (project_id, user_id) -> joined_at | BCNF |
| skill_exchanges | id | id -> all columns | BCNF |
| notifications | id | id -> all columns | BCNF |

### First normal form (1NF)

Every column holds one atomic value and no table has repeating groups. Skills, goals, interests and links are stored as one row per fact instead of comma-separated lists. Enumerations are single values. Every table has a primary key.

### Second normal form (2NF)

2NF only matters for composite keys: user_skills, user_learning_goals, user_interests, user_links, project_skills and project_members. In each, every non-key column depends on the whole key. For example, level needs both the user and the skill; joined_at needs both the project and the user. No column depends on part of a key, so there is no partial dependency.

### Third normal form (3NF)

No non-key column depends on another non-key column. Every transitive chain was removed:

- user -> college_id -> college name: college name lives in colleges.
- project -> category_id -> category name and skill -> category_id -> category name: category names live in lookup tables.

- notification -> application_id -> project_id: notifications store only the application or exchange id; the project is found by join.
- Team size, 'project is full', unread count, match score and notification text are derived values and are not stored at all.
- The 'decided by' column was dropped from applications because it is always projects.owner_id.

### Boyce-Codd normal form (BCNF)

Every determinant of every non-trivial dependency is a candidate key (see the table above). Two tables have two candidate keys: users (id and email) and project_applications (id and the pair project_id, applicant_id). In both, each key determines all other columns, so neither creates a determinant that is not a key. All tables are therefore in BCNF.

### Fourth normal form (4NF)

4NF removes independent multivalued facts that share a table. For one user, the skills they know, the skills they want to learn, their interests and their links are mutually independent. A single table Profile(user, known_skill, wanted_skill, interest) would need the cross product of rows (3 known x 2 wanted x 4 interests = 24 rows for 9 facts) and would have the multivalued dependencies user ->> known_skill | wanted_skill | interest. SkillBridge stores each fact set in its own table (user_skills, user_learning_goals, user_interests, user_links), so no table has a non-trivial multivalued dependency that is not implied by a key. The same reasoning separates project_skills (what a project needs) from project_members (who is on the team).

### Fifth normal form (5NF / project-join normal form)

5NF requires that every join dependency is implied by the candidate keys, i.e. no table can be split into smaller tables and joined back without loss unless the key already forces it. Two checks:

**1. The matching result is a join, so it is never stored.** Suppose a table Candidate(user, project, skill) recorded that a user can fill a skill needed by a project. It equals user_skills JOIN project_skills JOIN project_applications projected on (user, project, skill), a cyclic three-way join dependency. Storing it would violate 5NF and cause anomalies (changing one user's skill would require edits in many rows). SkillBridge keeps the three binary tables and computes the join in the matching query.

**2. skill_exchanges is a four-way fact that cannot be decomposed.** Try splitting it into (sender, recipient), (sender, offered_skill), (recipient, requested_skill) and (offered_skill, requested_skill). Joining these back creates spurious tuples: if Aarav offers React to Priya and Figma to Meera, the join also produces 'Aarav offers Figma to Priya', which never happened. The join dependency does not hold, so no lossless decomposition exists. The only key is the surrogate id (assumption A6), so the table is already in 5NF.

**Deliberate modelling choice.** The owner is stored once, in projects.owner_id, and is not duplicated in project_members. Seats taken = 1 + COUNT(project_members). Storing the owner in both places would create a redundancy that can drift out of sync.

## 5.7 Why some obvious columns are missing

| Column not stored | Reason | How it is obtained |
|---|---|---|
| projects.current_team_size | Derived; would need updating on every membership change (update anomaly, 3NF) | 1 + COUNT(project_members) |
| projects.status = FULL | Derived from capacity; could contradict the member count | Computed flag in queries |
| notifications.message | Derived from type and the referenced row | Rendered in the application |
| notifications.project_id | Determined by application_id (transitive dependency) | Join through project_applications |
| project_applications.decided_by | Always the project owner | Join to projects.owner_id |
| Match score | Depends on current profiles; stale as soon as a profile changes | Computed on request |

## 5.8 Transactions and concurrency

The critical multi-step operation is accepting an application: read the application, check ownership, check project status and capacity, insert the membership, update the application and insert the notification. All steps run in one database transaction. PostgreSQL's default isolation level (READ COMMITTED) allows two concurrent requests to both read 'one seat left'. The fix is to lock the project row with `SELECT ... FOR UPDATE` at the start of the transaction and to re-read the application status and member count after the lock is acquired. The second request then waits, sees the first request's membership row and receives a 409 Conflict. An alternative is SERIALIZABLE isolation with retry on serialisation failure. The sequence is shown in Figure 7.4.

## 5.9 Analytics view for the Data Engineering component

A SQL view summarises skill demand and supply. It is also used to demonstrate indexes with EXPLAIN ANALYZE during the code defence.

```
CREATE VIEW skill_demand_supply AS
SELECT s.id, s.name,
  (SELECT COUNT(*) FROM project_skills ps JOIN projects p ON p.id = ps.project_id
    WHERE ps.skill_id = s.id AND p.status = 'OPEN')                    AS open_projects_needing_it,
  (SELECT COUNT(*) FROM user_skills us WHERE us.skill_id = s.id)       AS students_who_know_it,
  (SELECT COUNT(*) FROM user_learning_goals g WHERE g.skill_id = s.id) AS students_who_want_it
FROM skills s;
```

## 5.10 Seed data plan

A seed script loads the skill catalogue (about 60 skills in 6 categories with aliases), project categories, interests and colleges, plus demo accounts and projects for the live demonstration and for load-testing the list query. Seed data is clearly marked as demo data.

# 6. Data flow diagrams

## 6.1 Notation

Rectangles are external entities (people outside the system). Ellipses are processes that transform data. Open-ended boxes labelled D1 to D6 are data stores (groups of database tables). Arrows are data flows and are labelled with the data they carry. A project owner is a student acting in the owner role, so there is no separate account type; the entity is drawn separately only to make the flows clear.

## 6.2 Level 0: context diagram

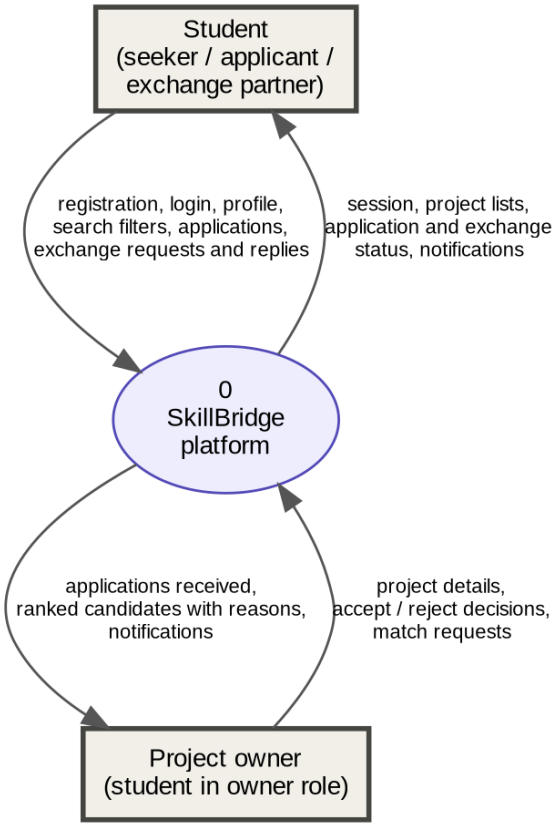



*Figure 6.1: Context diagram. The whole platform is one process exchanging data with two external entities.*

## 6.3 Level 1: main processes and data stores

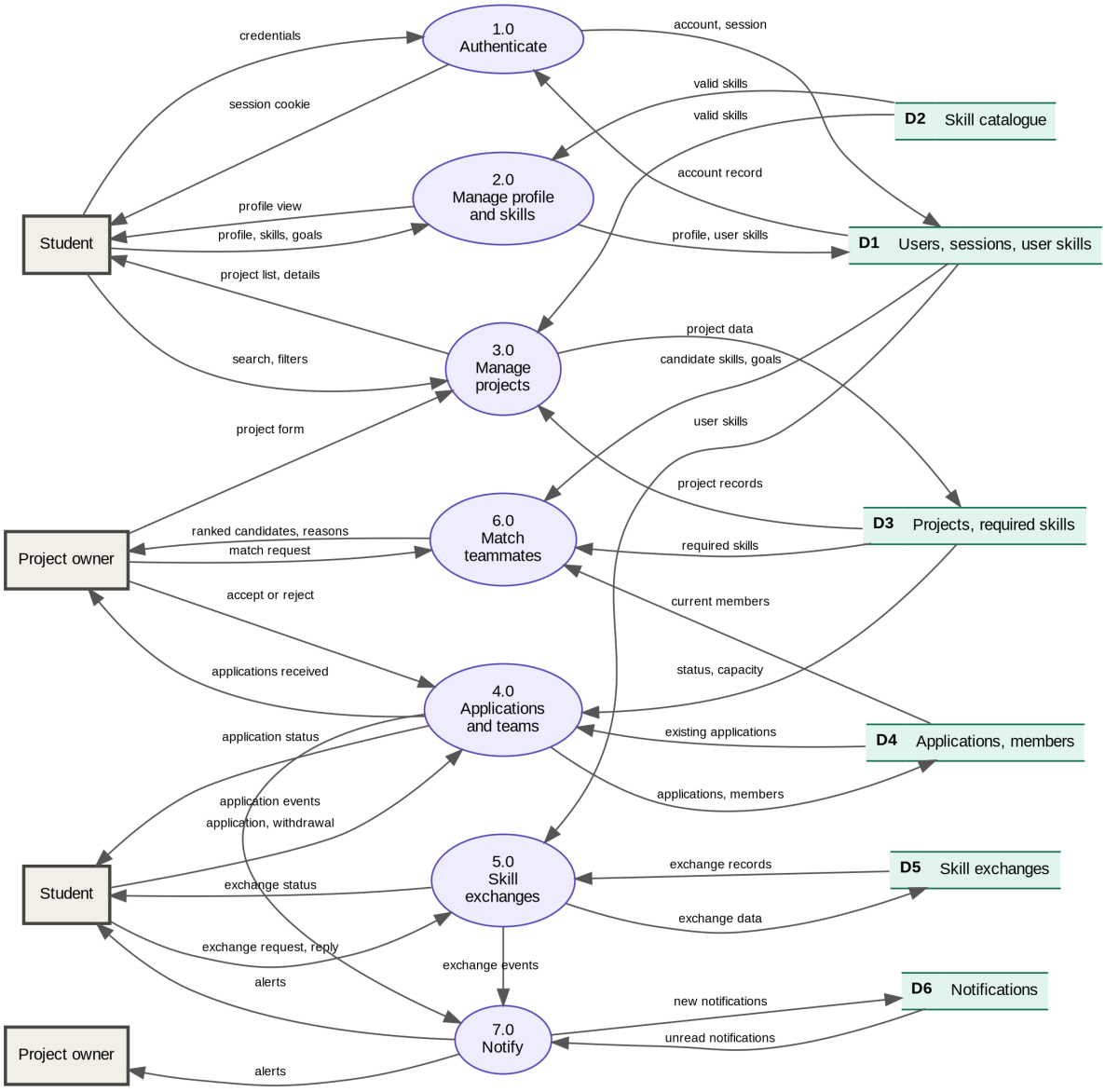



*Figure 6.2: Level 1 DFD. Entities are repeated to keep flows short.*

| Data store | Contains (database tables) | Used by processes |
|---|---|---|
| D1 Users, sessions, user skills | users, user_links, sessions, user_skills, user_learning_goals, user_interests, colleges | 1.0, 2.0, 5.0, 6.0 |
| D2 Skill catalogue | skills, skill_categories, skill_aliases, interests | 2.0, 3.0 |
| D3 Projects, required skills | projects, project_skills, project_categories | 3.0, 4.0, 6.0 |
| D4 Applications, members | project_applications, project_members | 4.0, 6.0 |
| D5 Skill exchanges | skill_exchanges | 5.0 |
| D6 Notifications | notifications | 7.0 |

## 6.4 Level 2: process 4.0 (applications and teams)

Process 4.0 is decomposed because it carries the most business rules. It has two independent flows: submitting an application (4.1 to 4.4) and deciding on it (4.5 to 4.8).

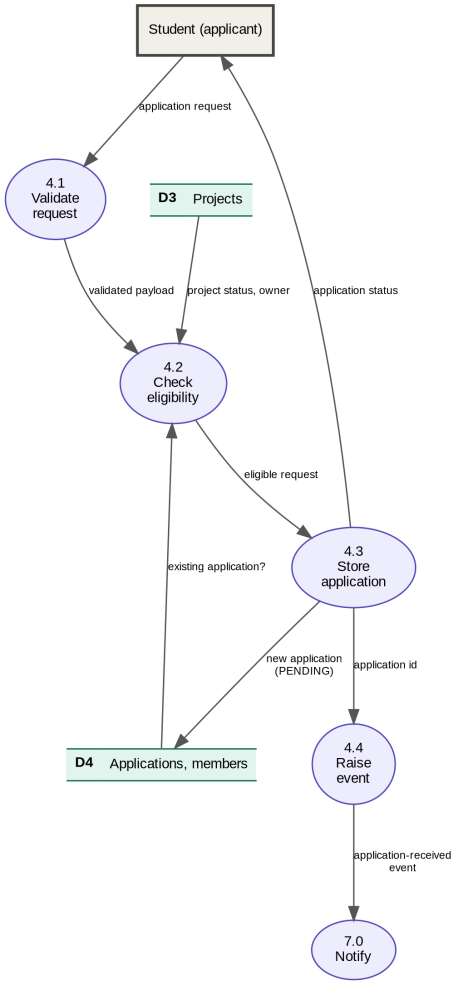



*Figure 6.3: Level 2, submitting an application (processes 4.1 to 4.4).*

| Process | Input | Reads / writes | Output |
|---|---|---|---|
| 4.1 Validate request | message, project id | none | validated payload, or 400 |
| 4.2 Check eligibility | applicant id, project id | reads D3 (status, owner), D4 (existing application) | eligible request, or 400 / 409 |
| 4.3 Store application | eligible request | writes D4 (status PENDING) | application status to the student |
| 4.4 Raise event | application id | to process 7.0 | 'application received' notification for the owner |

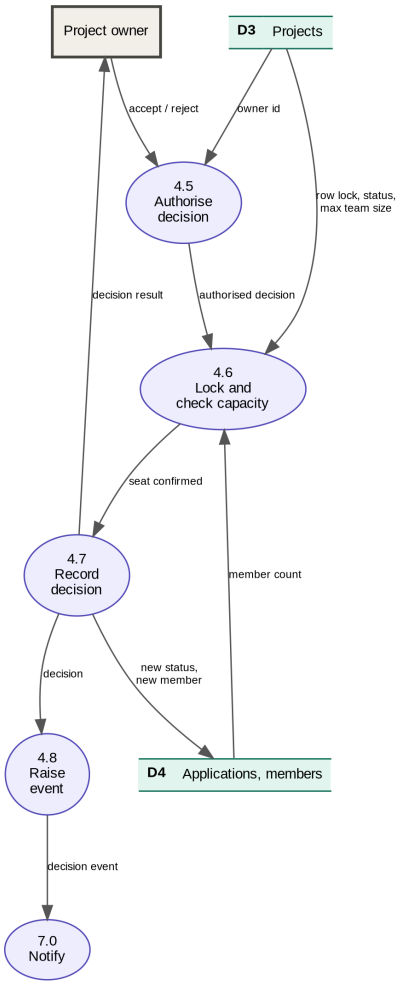



*Figure 6.4: Level 2, deciding on an application (processes 4.5 to 4.8).*

| Process | Input | Reads / writes | Output |
|---|---|---|---|
| 4.5 Authorise decision | decision, caller id | reads D3 (owner id) | authorised decision, or 403 |
| 4.6 Lock and check capacity | project id | locks the D3 project row; reads D4 member count | seat confirmed, or 409 (full / not open / already decided) |
| 4.7 Record decision | decision | writes D4 (new status and, if accepted, the new member) | decision result to the owner |
| 4.8 Raise event | decision | to process 7.0 | 'accepted' or 'rejected' notification for the applicant |

Steps 4.6, 4.7 and the notification write run inside one database transaction.

# 7. Low-level design

## 7.1 Request lifecycle

Every API call travels through the same pipeline: `helmet, cors, pino-http, json (size limit), cookieParser, rateLimit, route, requireAuth, validate (zod), controller, service, Prisma, PostgreSQL`, and any error is caught by one central error handler. Example: `POST /api/projects/:id/applications`.

1. The React API client sends the request with the session cookie (withCredentials).
2. **requireAuth** hashes the cookie token, looks up the session, rejects missing or expired sessions with 401 and sets req.user.
3. **validate** checks the project id (UUID) and the message (10 to 500 characters) with a zod schema; failure returns 400 with field errors.
4. The **controller** calls applicationsService.applyToProject(projectId, user.id, message) and shapes the HTTP response.
5. The **service** loads the project, rejects the owner (400), non-open projects (409) and duplicates (409), then creates the application and the owner's notification in one transaction.
6. Any thrown AppError, ZodError or known Prisma error is converted by the **error handler** into a consistent JSON error and logged.

## 7.2 Backend structure and conventions

```
skillbridge/
|-- client/                       React + Vite + Tailwind
|   `-- src/ api/ components/ features/{auth,profile,projects,applications,exchanges,matching}/
|            hooks/ layouts/ pages/ routes/ utils/ App.jsx main.jsx
|-- server/
|   |-- prisma/ schema.prisma  migrations/  seed.js
|   |-- src/
|   |   |-- config/        env loading and validation, prisma client, logger
|   |   |-- middleware/    requireAuth, validate, errorHandler, rateLimiters
|   |   |-- modules/       auth users skills projects applications exchanges matching notifications
|   |   |                  (each: *.routes.js *.schema.js *.controller.js *.service.js *.test.js)
|   |   |-- utils/         errors.js, asyncHandler.js
|   |   |-- app.js         builds the Express app (used by tests)
|   |   `-- server.js      starts the HTTP server
|   `-- tests/             shared test helpers, test database setup
|-- docs/                  ERD, DFD, LLD, API reference, this document
|-- .github/workflows/     CI pipeline
|-- .env.example  .gitignore  README.md  package.json
```

## 7.3 Module catalogue

| Module | Main service functions | Tables touched |
|---|---|---|
| auth | register, login, logout, getCurrentUser | users, sessions |
| users | getPublicProfile, updateProfile, replaceSkills, replaceLearningGoals, replaceInterests, replaceLinks | users, user_skills, user_learning_goals, user_interests, user_links |
| skills | searchSkills, resolveSkillName (alias lookup) | skills, skill_aliases, skill_categories |
| projects | createProject, listProjects, getProject, updateProject, changeStatus | projects, project_skills |
| applications | applyToProject, listForProject, decideApplication, withdrawApplication, listMine | project_applications, project_members, notifications |
| exchanges | createExchange, listMine, changeExchangeStatus | skill_exchanges, notifications |
| matching | getMatchesForProject, scoreCandidate (pure function) | read only: project_skills, user_skills, user_learning_goals, project_members |
| notifications | list, markRead, markAllRead, unreadCount | notifications |

## 7.4 REST API

### Conventions

All responses are JSON. Success: `{ "data": ... }`. Failure: `{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": { "field": ["..."] } } }`. Lists accept `page` and `limit` (limit capped at 50 on the server). Where revealing that a resource exists would leak information (for example another user's notification), the API returns 404 instead of 403.

| Status | Used for |
|---|---|
| 200 / 201 / 204 | Successful read or update / resource created / success without body (logout) |
| 400 | Validation failure or invalid operation (for example applying to own project) |
| 401 | Missing, invalid or expired session; wrong login credentials |
| 403 | Authenticated but not allowed (for example editing someone else's project) |
| 404 | Resource does not exist or is not visible to the caller |
| 409 | Conflict: duplicate, full team, already processed, illegal state transition |
| 429 | Rate limit exceeded |
| 500 | Unexpected server error; details only in logs |

### Endpoint catalogue (planned)

| Method and path | Auth | Authorization rule | Success | Errors |
|---|---|---|---|---|
| POST /api/auth/register | No | None | 201 | 400, 409 |
| POST /api/auth/login | No | None; rate-limited | 200 | 400, 401, 429 |
| POST /api/auth/logout | Yes | Own session | 204 | 401 |
| GET /api/auth/me | Yes | Own account | 200 | 401 |
| GET /api/users/:id | Yes | Public fields only | 200 | 404 |
| PATCH /api/users/me | Yes | Own profile | 200 | 400 |
| PUT /api/users/me/skills | Yes | Own skills; names resolved through the catalogue | 200 | 400, 409 |
| GET /api/skills?query= | Yes | None | 200 | 400 |
| GET /api/projects | Yes | Only OPEN / IN_PROGRESS unless the caller is owner | 200 | 400 |
| POST /api/projects | Yes | Any student | 201 | 400 |
| GET /api/projects/:id | Yes | DRAFT visible only to owner | 200 | 404 |
| PATCH /api/projects/:id | Yes | Owner only; status transition must be allowed | 200 | 400, 403, 409 |
| POST /api/projects/:id/applications | Yes | Not the owner; project OPEN; no duplicate | 201 | 400, 404, 409 |
| GET /api/projects/:id/applications | Yes | Owner only | 200 | 403 |
| PATCH /api/applications/:id | Yes | Owner: ACCEPTED or REJECTED. Applicant: WITHDRAWN | 200 | 403, 404, 409 |
| GET /api/users/me/applications | Yes | Own applications | 200 | 401 |
| POST /api/exchanges | Yes | Not self; offered skill on own profile; requested skill on recipient's profile | 201 | 400, 409 |
| GET /api/exchanges | Yes | Only exchanges the caller sent or received | 200 | 401 |
| PATCH /api/exchanges/:id | Yes | Per the exchange state machine | 200 | 403, 404, 409 |
| GET /api/projects/:id/matches | Yes | Owner only | 200 | 403 |
| GET /api/notifications | Yes | Own only | 200 | 401 |
| PATCH /api/notifications/:id/read | Yes | Own only (404 otherwise) | 200 | 404 |
| PATCH /api/notifications/read-all | Yes | Own only | 200 | 401 |

| Method and path | Auth | Authorization rule | Success | Errors |
|---|---|---|---|---|
| GET /api/health | No | None | 200 | none |

## 7.5 State machines

Every status field has an explicit transition table. Any transition not listed is rejected with 409, and each transition names the actor allowed to perform it.

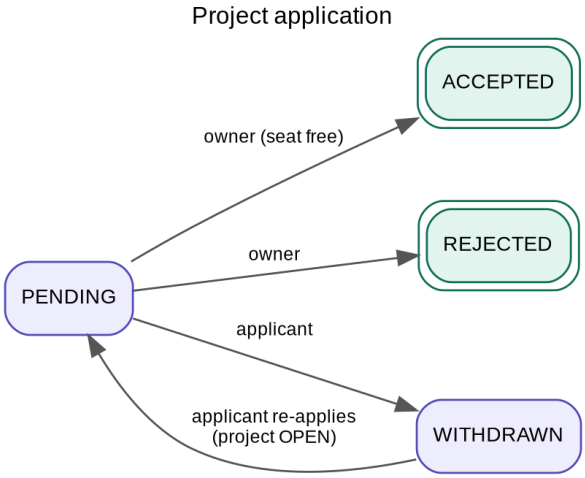



*Figure 7.1: Project application lifecycle (double border = final state).*

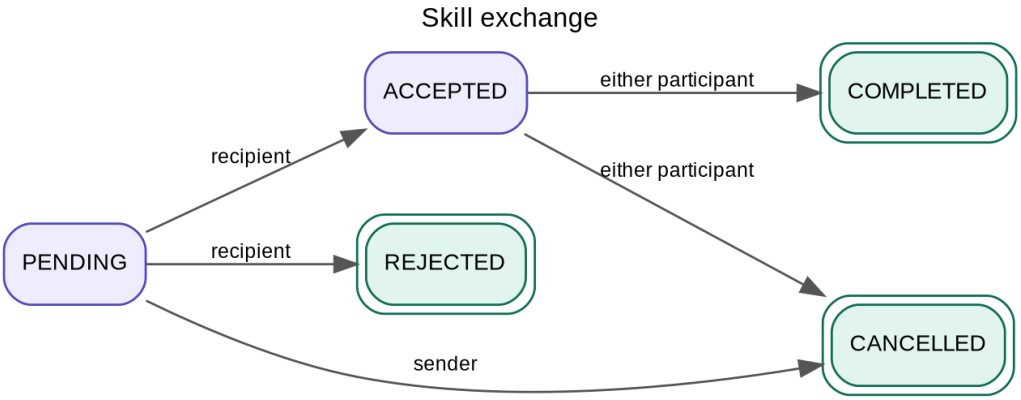



*Figure 7.2: Skill exchange lifecycle.*

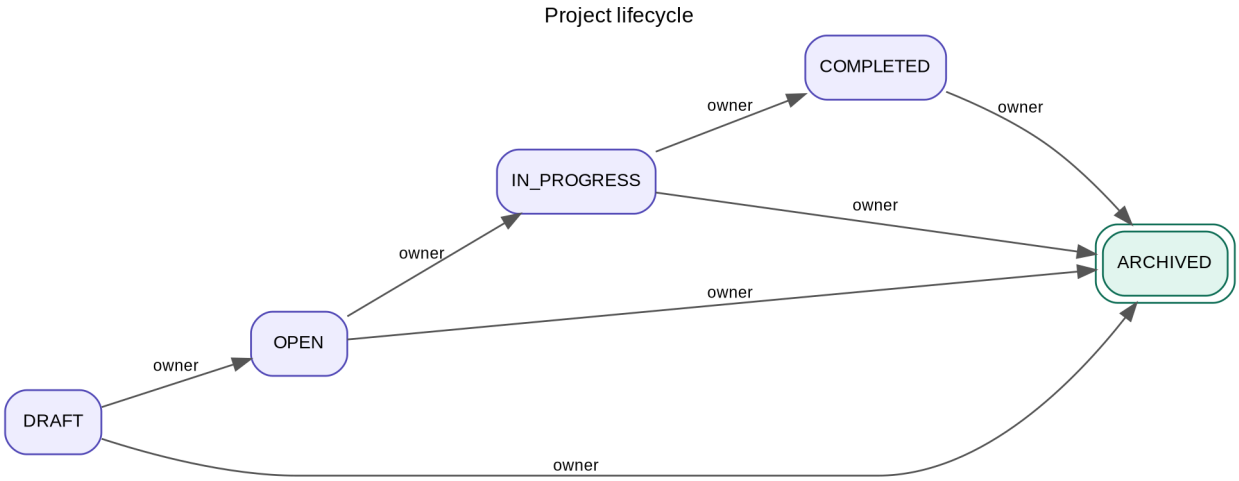



*Figure 7.3: Project lifecycle. 'Full' is not a state; it is computed from capacity. Applications are accepted only in OPEN.*

## 7.6 Key algorithms

### Applying to a project

```
applyToProject(projectId, applicantId, message):
    project = find project by id            -> not found: 404
    if project.ownerId == applicantId       -> 400 "cannot apply to own project"
    if project.status != OPEN               -> 409 "not accepting applications"
    begin transaction
        application = insert (projectId, applicantId, message, status PENDING)
        insert notification (recipient = project.ownerId, type APPLICATION_RECEIVED, applicationId)
    commit
    on unique-violation (projectId, applicantId)  -> 409 "already applied"
```

### Deciding on an application (concurrency-safe)

```
decideApplication(applicationId, ownerId, decision):          // decision = ACCEPTED or REJECTED
  transaction:
    application = find application with its project             -> not found: 404
    if application.project.ownerId != ownerId                  -> 403
    lock project row:  SELECT id FROM projects WHERE id = ? FOR UPDATE
    application = re-read inside the lock
    if application.status != PENDING                           -> 409 "already processed"
    if decision == ACCEPTED:
        project = re-read project with member count
        if project.status != OPEN                              -> 409 "project not open"
        if 1 + memberCount >= project.maxTeamSize              -> 409 "team is full"
        insert project_members (projectId, applicantId)
    update application: status = decision, decidedAt = now
    insert notification (recipient = applicant, type APPLICATION_ACCEPTED or APPLICATION_REJECTED)
  commit   // all writes succeed together or none do
```

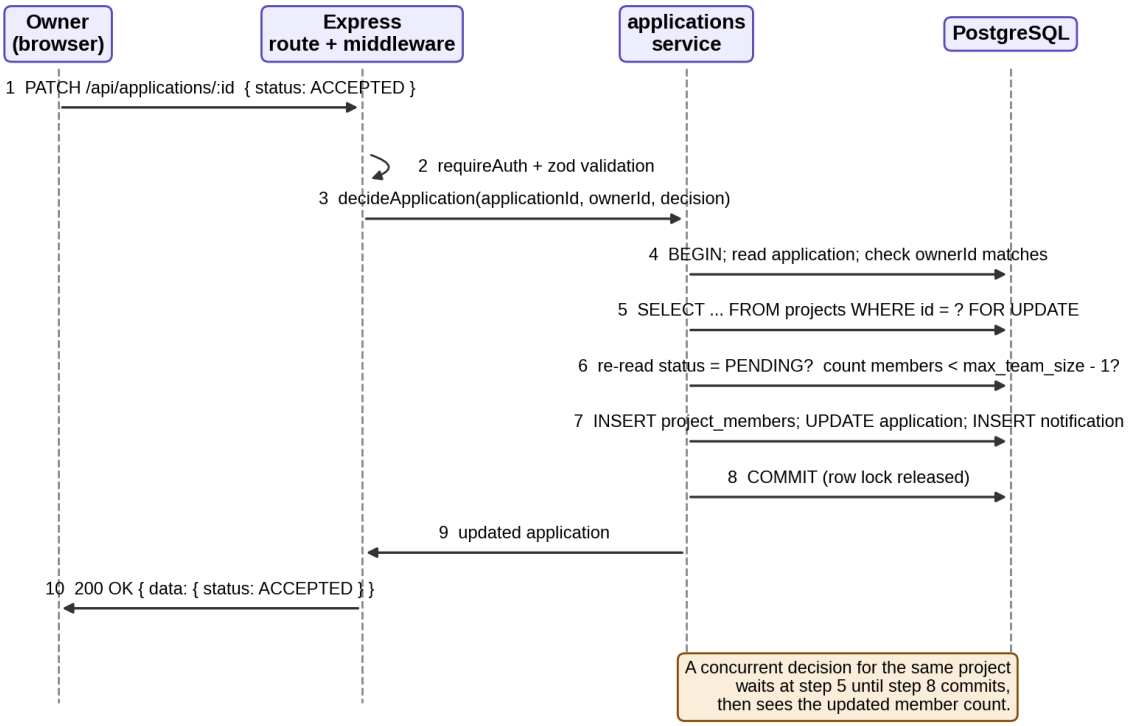



*Figure 7.4: Sequence of an accept decision. A concurrent request for the same project waits at the row lock.*

### Skill-exchange creation rules

```
createExchange(senderId, recipientId, offeredSkillId, requestedSkillId, message):
    if senderId == recipientId                                  -> 400
    if sender does not have offeredSkillId in user_skills       -> 400
    if recipient does not have requestedSkillId in user_skills  -> 400
    insert skill_exchange (status PENDING)
    on violation of the partial unique index (active duplicate)  -> 409
```

### Explainable teammate matching

The engine is a deterministic, rule-based scorer. It ranks candidates for a project and returns, for each, a score from 0 to 100, a component breakdown and human-readable reasons. The score is a ranking aid; it is not a probability of success, and the weights are initial design values to be tuned on test profiles.

| Component | Weight | Definition (each normalised to 0..100) |
|---|---|---|
| R: required-skill coverage | 0.60 | For each required skill: 1 point if the candidate knows it at or above the minimum level, 0.5 if below, 0 if unknown. R = points / number of required skills x 100. |
| P: proficiency fit | 0.20 | Average of (level / 3) over the required skills the candidate knows; levels BEGINNER = 1, INTERMEDIATE = 2, ADVANCED = 3. |
| C: gap filling | 0.10 | Share of required skills that nobody on the team has yet and that the candidate has. |
| L: learning alignment | 0.10 | Share of required skills the candidate wants to learn but does not know yet. |

**Score = 0.60 R + 0.20 P + 0.10 C + 0.10 L**, rounded to an integer.

### Worked example

Project requires React (minimum INTERMEDIATE), Node.js (INTERMEDIATE) and PostgreSQL (BEGINNER). The owner already covers Node.js and PostgreSQL. Candidate knows React (INTERMEDIATE), Node.js (BEGINNER) and Figma (INTERMEDIATE) and wants to learn PostgreSQL.

| Component | Calculation | Value |
|---|---|---|
| R | React 1 + Node.js 0.5 + PostgreSQL 0 = 1.5 of 3 | 50.0 |
| P | (2/3 + 1/3) / 2 matched skills | 50.0 |
| C | Uncovered required skills = {React}; candidate has it: 1 of 1 | 100.0 |
| L | PostgreSQL is wanted and unknown: 1 of 3 | 33.3 |
| Score | 0.60 x 50 + 0.20 x 50 + 0.10 x 100 + 0.10 x 33.3 | 53 |

Reasons shown to the owner: 'React: intermediate level meets the requirement', 'Node.js: known, but below the required intermediate level', 'Adds 1 skill the team does not have yet', 'Wants to learn PostgreSQL'. Candidates are pre-filtered in SQL (users who have or want at least one required skill, using the indexes on skill_id), the owner and current members are excluded, scoring runs in a pure function, and the top 20 are returned.

## 7.7 Error handling and logging

| Concern | Design |
|---|---|
| Error model | AppError(statusCode, code, message, details) with helpers badRequest, unauthorized, forbidden, notFound, conflict |
| Central handler | Converts AppError, ZodError (to 400 with field errors) and known Prisma errors (P2002 to 409, P2025 to 404); any other error becomes a generic 500 and is logged with the stack |
| Async safety | asyncHandler wrapper (or Express 5) forwards rejected promises to the handler |
| Structured logs | pino-http emits one JSON line per request with a request id; levels: info (requests), warn (expected 4xx), error (5xx) |
| Business events | Logged with ids only, for example 'application decided' with applicationId and decision |
| Redaction | Cookies and password fields are redacted before logging |
| Process safety | unhandledRejection and uncaughtException are logged, then the process exits so the host restarts it |

## 7.8 Frontend low-level design

| Concern | Design |
|---|---|
| Routing | React Router; ProtectedRoute redirects to /login when GET /api/auth/me returns 401 |
| Server state | TanStack Query hooks per feature (useProjects, useApplications, ...): caching, loading and error states, invalidation after mutations |
| Forms | react-hook-form with a zod resolver; messages come from the same rules as the server |

| Concern | Design |
|---|---|
| API client | One axios instance with withCredentials; interceptor maps error.code to user-friendly toasts and redirects on 401 |
| Auth state | AuthProvider context holds the current user from /api/auth/me |
| Every page | Loading skeleton, empty state ('No projects match your filters' with a clear-filters button), error banner, disabled buttons while submitting, confirmation before destructive actions |

| Route | Page | Access |
|---|---|---|
| /login, /register | Authentication forms | Public |
| / | Dashboard: recommended projects, pending applications, incoming exchanges, notifications | Logged in |
| /projects, /projects/new, /projects/:id, /projects/:id/edit | Discovery with filters, create, details, edit | Logged in; edit only for owner |
| /projects/:id/applications, /projects/:id/matches | Review queue and ranked candidates | Owner only |
| /users/:id, /profile/edit | Public profile, edit profile and skills | Logged in; edit only own |
| /applications, /exchanges, /notifications | My applications, exchanges, notification list | Logged in |

Key components: AuthProvider, ProtectedRoute, NavBar with NotificationBell, ProjectCard, ProjectFilters, SkillPicker (autocomplete over the catalogue), ApplyDialog, ApplicationList, MatchCard (score and reasons), ExchangeForm, EmptyState, ErrorBanner.

# 8. Security design

## 8.1 Threats and controls

| Threat | Control |
|---|---|
| Password theft | bcrypt with cost 12; password hash never returned; generic login error message |
| Session theft | HTTP-only cookie, Secure in production, 7-day expiry; only the SHA-256 hash of the token is stored; logout deletes the session |
| CSRF | SameSite cookie policy plus Origin header check on state-changing requests (CSRF token if cross-site cookies are needed) |
| Broken access control | Ownership and membership checked inside every service function, not in the UI (changing /projects/123 to /projects/456 must fail) |
| Brute force and abuse | express-rate-limit on login and register; request size limit |
| SQL injection | Prisma parameterised queries; the only raw SQL uses parameterised tagged templates |
| Mass assignment | zod schemas whitelist accepted fields, so clients cannot set ownerId, status or id |
| Data leakage | Explicit field selection for public profiles; 404 for resources the caller may not know about |
| Misconfiguration | helmet headers, narrow CORS origin, secrets only in environment variables, .env in .gitignore, .env.example committed |
| Vulnerable dependencies | Maintained packages, lockfile committed, npm audit in CI |

## 8.2 Permission matrix

| Operation | Visitor | Student | Project owner | Applicant | Exchange recipient |
|---|---|---|---|---|---|
| Register, log in | Yes | n/a | n/a | n/a | n/a |
| Browse projects and public profiles | No | Yes | Yes | Yes | Yes |
| Edit own profile and skills | No | Yes | Yes | Yes | Yes |
| Create a project | No | Yes | Yes | Yes | Yes |
| Edit or archive a project | No | No | Own project only | No | No |
| Apply to a project | No | Yes (not own, open only) | No (own) | Yes | Yes |
| Accept or reject an application | No | No | Own project only | No | No |
| Withdraw an application | No | No | No | Own pending only | No |
| View match list for a project | No | No | Own project only | No | No |
| Accept or reject an exchange | No | No | No | No | Yes (pending, addressed to them) |
| Read notifications | No | Own only | Own only | Own only | Own only |

# 9. Quality assurance and testing strategy

## 9.1 Test levels

| Level | Tools | What is tested |
|---|---|---|
| Unit | Vitest | scoreCandidate with fixed profiles; status-transition tables; validation schemas |
| Integration (API) | Vitest, Supertest, separate test database | Full request pipeline: authentication, authorisation, business rules, database constraints, error format |
| End to end (if time allows) | Playwright | Register, create project, apply, accept, exchange |
| Static checks | ESLint, Prettier | Style and common bugs, enforced in CI |

## 9.2 Test case matrix

| Scenario | Expected result | Level |
|---|---|---|
| Register with an existing email | 409 with a useful message | Integration |
| Login with wrong password | 401 with a generic message | Integration |
| Call a protected endpoint without a session | 401 | Integration |
| Edit another user's project | 403 | Integration |
| Apply twice to the same project | Second request returns 409 | Integration |
| Owner applies to own project | 400 | Integration |
| Apply to a full or non-open project | 409 | Integration |
| Fire two accept requests for the last seat with Promise.all | Exactly one succeeds, one gets 409, members never exceed the limit | Integration (concurrency) |
| Accept an application | Membership row, application status and notification all exist | Integration |
| Decide an application twice | Second request returns 409 | Integration |
| Send an exchange to yourself | 400 (and the CHECK constraint blocks it even if the service is bypassed) | Integration |
| Duplicate active exchange | 409 from the partial unique index | Integration |
| Read another user's notification | 404 | Integration |
| Search that matches nothing | 200 with an empty list | Integration |
| Score for the worked example in Section 7.6 | 53 with the four reasons listed | Unit |

## 9.3 Continuous integration

A GitHub Actions workflow runs on every push and pull request: install dependencies, lint, start a PostgreSQL service container, apply migrations, run the test suite and build the client. A failing check blocks merging into the main branch.

# 10. Deployment and operations plan

Live cloud deployment is worth one mark in Stage 2 and a localhost demo forfeits it, so deployment is planned from the start and a minimal 'hello world' version (health endpoint plus database connection) will be deployed at the end of the first build week to find hosting and cookie problems early. Hosting providers are chosen at that point because free-tier terms and limits change; the team will verify them then.

## 10.1 Configuration (environment variables)

| Variable | Purpose | Notes |
|---|---|---|
| DATABASE_URL | PostgreSQL connection string | Never committed |
| NODE_ENV | development, test or production | Controls Secure cookies and log format |
| PORT | API port | Provided by the host in production |
| CLIENT_ORIGIN | Allowed CORS origin and Origin-check value | Exact URL of the deployed frontend |
| SESSION_DAYS | Session lifetime | Default 7 |
| LOG_LEVEL | pino log level | info in production |
| VITE_API_URL | API base URL for the client build | Public value, no secrets |

A committed `.env.example` lists every variable with a dummy value; the real `.env` files are in .gitignore. The server validates required variables at start-up and refuses to run if one is missing.

## 10.2 Release checklist

| # | Check |
|---|---|
| 1 | Production environment variables are set on the host; no secret is in Git. |
| 2 | Database reachable from the API; `prisma migrate deploy` applied, including the raw SQL constraints. |
| 3 | Seed script loaded the skill catalogue and demo data. |
| 4 | Frontend built with the production API URL; HTTPS enabled everywhere. |
| 5 | CORS and cookie settings match the deployment layout (same site or cross-site). |
| 6 | GET /api/health returns 200; logs are visible in the host's log viewer. |
| 7 | Registration, login, project creation, application and acceptance verified on the deployed version. |
| 8 | README contains the live URL, setup steps and demo credentials policy. |

Backups and recovery: for any real production use the managed database's backup feature would be enabled; for this academic project the team will export a schema and seed dump before the final demonstration.

# 11. Project plan, team and risks

## 11.1 Timeline

| Dates (2026) | Phase | Goals and deliverables |
|---|---|---|
| 13 to 24 Oct | Stage 1 (evaluation window) | This design document; public repository initialised with README and /docs; Express skeleton with health endpoint and first migration; 15-minute presentation |
| 26 Oct to 1 Nov | Build week 1: foundation | Environment, schema migration with raw SQL constraints, seed script, authentication and sessions, profile basics, CI, early hello-world deployment |
| 2 to 8 Nov | Build week 2: core workflows | Profiles and skills, projects with discovery and filters, applications and team formation with row locking, skill exchanges, API tests |
| 9 to 15 Nov | Build week 3: integration | Matching engine, notifications, dashboard, all pages connected to the API, loading / empty / error / mobile states |
| 16 to 22 Nov | Build week 4: quality and release | Permission and concurrency tests, security review, accessibility, production deployment, README and API documentation |
| 23 to 25 Nov | Stabilise | Bug fixes only, demo data, rehearsal of the demonstration scenario and code defence |
| 26 Nov to 4 Dec | Stage 2 (evaluation window) | Full integration, live deployment, presentation and code defence |

The plan is a target, not a guarantee. If work slips, the essential workflows (authentication, projects, applications and teams, exchanges, matching) are protected and optional items (dashboard polish, analytics view, end-to-end browser tests) are cut first.

## 11.2 Team and responsibilities (proposed)

| Area | Primary | Reviewer |
|---|---|---|
| Database schema, migrations, seed, SQL constraints, backend services, backend tests | Member 1 | Member 2 |
| React application, forms, state, responsive UI, accessibility | Member 2 | Member 1 |
| API contracts, authentication flow, error format | Both (agreed before coding) | Both |
| Deployment, CI, README, documentation | Both (split by task) | Both |

Both members must be able to explain every part of the system during the code defence, so every pull request is reviewed by the other member.

## 11.3 Risk register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Scope grows beyond what two people can finish | High | High | Scope frozen to the MVP in Section 2.6; out-of-scope list in Section 2.7; weekly scope check |
| Concurrency bug in team capacity | Medium | High | Row lock plus re-check; dedicated Promise.all test; explained in the defence |
| Cookie or CORS problems after deployment | Medium | High | Decide same-site versus cross-site early; deploy a skeleton in build week 1 |
| Free hosting limits or cold starts | Medium | Medium | Verify terms when choosing; keep the app light; warm the service before demos |
| One member unavailable | Low | High | Shared ownership, pull-request reviews, documented decisions |
| Code written but not understood | Medium | High | Write modules by hand from the LLD; explain each module to the other member; practise the defence |

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Late discovery of schema problems | Medium | Medium | Schema reviewed against every API in this document before the first migration; changes only through new migrations |

# 12. GitHub repository and documentation plan

Stage 1 requires an initialised public GitHub repository with project setup and initial documentation. The handbook also asks for clean structure, proper naming, a clear commit history and a professional README.

## 12.1 Repository conventions

| Item | Convention |
|---|---|
| Name and visibility | skillbridge, public |
| Branches | main is protected and always deployable; work happens on feature/<module>-<topic> branches merged by pull request after review and a passing CI run |
| Commit messages | Conventional Commits: feat:, fix:, docs:, test:, refactor:, chore: followed by a short imperative summary |
| Issues and board | One issue per feature ID (F-01 to F-12) on a GitHub Projects board: Backlog, In progress, Review, Done |
| Pull requests | Template with description, linked issue, test evidence and a checklist (lint, tests, docs updated) |
| Secrets | .env in .gitignore; .env.example committed; no credentials in history |

## 12.2 Initial repository contents (Stage 1)

| Path | Content |
|---|---|
| README.md | Project title, problem statement, feature list, tech stack, architecture diagram, setup instructions (prerequisites, environment, migrate, seed, run), status badge, team, licence, acknowledgements and AI usage |
| docs/ | This design document (PDF), ERD image and Prisma schema, DFD images, LLD summary, API reference (to grow), decisions log |
| server/ | Express skeleton: app.js, server.js, health endpoint, error handler, logger, Prisma schema and first migration |
| client/ | Vite React skeleton with routing and a placeholder home page |
| .github/workflows/ci.yml | Lint and test pipeline |
| .env.example, .gitignore, LICENSE | Configuration template, ignore rules, licence |

## 12.3 Stage 1 repository checklist

| | Item |
|---|---|
| [ ] | Public repository created and the URL written on the cover page of this document |
| [ ] | README with problem statement, features, stack, architecture and setup steps committed |
| [ ] | /docs contains the ERD, DFDs, Prisma schema and this document |
| [ ] | Server skeleton runs locally; GET /api/health returns 200 |
| [ ] | First Prisma migration (including raw SQL constraints) applies cleanly to an empty database |
| [ ] | Commit history is clean and uses the agreed message format |
| [ ] | Acknowledgements and AI usage section present in the README |

# 13. Mapping to the evaluation framework

## 13.1 Stage 1 (13 to 24 October)

| Evaluation item | How this project satisfies it |
|---|---|
| Problem statement and ideation | Section 2: problem, solution, personas, features, requirements, business rules |
| End goal definition | Section 3: deployed responsive website with an acceptance checklist and demo scenario |
| Tech stack architecture and justification | Section 4: layered architecture, stack table with alternatives, key decisions |
| Database diagrams (ER diagram for SQL) | Section 5 and Appendix A: ERD, data dictionary, constraints, indexes, normalization from 1NF to 5NF |
| Data flow diagram and low-level design (faculty request) | Sections 6 and 7 |
| Initialised public GitHub repository | Section 12: conventions, contents and checklist |

## 13.2 Stage 2 (26 November to 4 December), planned

| Evaluation item | Planned approach |
|---|---|
| Full backend integration | Frontend, API and database connected for all seven modules (Sections 4 and 7) |
| Live cloud deployment (1 mark) | Hosted frontend, API and managed PostgreSQL; early skeleton deployment (Section 10) |
| Presentation and code defence | Architecture, decisions and live demo (Section 3.3); both members can explain every module |
| User authentication and authorisation | bcrypt, server-side sessions, ownership checks in services (Sections 7 and 8) |
| Structured exception handling | AppError hierarchy and central error handler (Section 7.7) |
| Application logging | pino structured logs with request ids and redaction (Section 7.7) |
| Professional repository | Conventions in Section 12; CI on every push |
| Resource attribution | Section 14 and the README |

The Advanced Programming small project (Bucket 1) and the coding contests are separate components and are not covered by this document.

# 14. Resource attribution and academic integrity

The handbook requires external libraries, open-source code, frameworks and AI tools to be cited in the repository README. The planned dependencies are listed below; exact versions and licences will be taken from package.json and verified at install time.

| Component | Licence (expected) | Component | Licence (expected) |
|---|---|---|---|
| React, React Router | MIT | Express, helmet, cors, express-rate-limit | MIT |
| Vite, Vitest | MIT | Prisma | Apache-2.0 |
| Tailwind CSS | MIT | PostgreSQL | PostgreSQL Licence |
| TanStack Query, react-hook-form, axios | MIT | zod, pino, bcrypt, Supertest | MIT |
| Playwright (optional) | Apache-2.0 | Graphviz (used to draw this document's diagrams) | EPL |

**AI usage declaration.** An AI assistant (Claude, by Anthropic) was used to discuss and structure the design and to help draft this document and its diagrams. The team reviewed the design and is responsible for its content. Any further AI assistance during implementation will be declared in the repository README. All implementation code will be written and understood by the team, and no uncredited code will be used.

# Appendix A. Prisma schema (source of the ERD)

This schema is the machine-readable form of Figure 5.1. It can be rendered to an image with prisma-erd-generator or pasted into a diagram tool.

```
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"  url = env("DATABASE_URL") }

enum Level             { BEGINNER INTERMEDIATE ADVANCED }
enum ProjectStatus     { DRAFT OPEN IN_PROGRESS COMPLETED ARCHIVED }
enum ApplicationStatus { PENDING ACCEPTED REJECTED WITHDRAWN }
enum ExchangeStatus    { PENDING ACCEPTED REJECTED CANCELLED COMPLETED }
enum LinkType          { GITHUB LINKEDIN PORTFOLIO OTHER }
enum NotificationType {
  APPLICATION_RECEIVED APPLICATION_ACCEPTED APPLICATION_REJECTED
  EXCHANGE_RECEIVED EXCHANGE_ACCEPTED EXCHANGE_REJECTED EXCHANGE_COMPLETED
}

model College {
  id    Int    @id @default(autoincrement())
  name  String @unique
  users User[]
  @@map("colleges")
}

model User {
  id           String   @id @default(uuid())
  email        String   @unique
  passwordHash String   @map("password_hash")
  fullName     String   @map("full_name")
  bio          String?
  weeklyHours  Int?     @map("weekly_hours")
  collegeId    Int?     @map("college_id")
  createdAt    DateTime @default(now()) @map("created_at")
  updatedAt    DateTime @updatedAt @map("updated_at")
  college           College?            @relation(fields: [collegeId], references: [id])
  links             UserLink[]
  sessions          Session[]
  skills            UserSkill[]
  learningGoals     UserLearningGoal[]
  interests         UserInterest[]
  ownedProjects     Project[]           @relation("ProjectOwner")
  applications      ProjectApplication[]
  memberships       ProjectMember[]
  sentExchanges     SkillExchange[]     @relation("ExchangeSender")
  receivedExchanges SkillExchange[]     @relation("ExchangeRecipient")
  notifications     Notification[]
  @@map("users")
}

model UserLink {
  userId String   @map("user_id")
  type   LinkType @map("link_type")
  url    String
  user   User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@id([userId, type])
  @@map("user_links")
}

model Session {
  id        String   @id   // SHA-256 hash of the cookie token, never the raw token
  userId    String   @map("user_id")
  expiresAt DateTime @map("expires_at")
  createdAt DateTime @default(now()) @map("created_at")
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@index([userId])
  @@index([expiresAt])
  @@map("sessions")
}

model SkillCategory {
  id     Int     @id @default(autoincrement())
  name   String  @unique
  skills Skill[]
  @@map("skill_categories")
}

model Skill {
  id         Int           @id @default(autoincrement())
  name       String        @unique
  categoryId Int           @map("category_id")
  category   SkillCategory @relation(fields: [categoryId], references: [id])
  aliases            SkillAlias[]
  userSkills         UserSkill[]
  learningGoals      UserLearningGoal[]
```

```
  projectSkills      ProjectSkill[]
  offeredExchanges   SkillExchange[] @relation("OfferedSkill")
  requestedExchanges SkillExchange[] @relation("RequestedSkill")
  @@map("skills")
}

model SkillAlias {
  alias   String @id   // lower case, e.g. "nodejs" maps to Node.js
  skillId Int    @map("skill_id")
  skill   Skill  @relation(fields: [skillId], references: [id], onDelete: Cascade)
  @@map("skill_aliases")
}

model UserSkill {
  userId  String @map("user_id")
  skillId Int    @map("skill_id")
  level   Level
  user    User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  skill   Skill  @relation(fields: [skillId], references: [id])
  @@id([userId, skillId])
  @@index([skillId])
  @@map("user_skills")
}

model UserLearningGoal {
  userId  String @map("user_id")
  skillId Int    @map("skill_id")
  user    User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  skill   Skill  @relation(fields: [skillId], references: [id])
  @@id([userId, skillId])
  @@index([skillId])
  @@map("user_learning_goals")
}

model Interest {
  id    Int            @id @default(autoincrement())
  name  String         @unique
  users UserInterest[]
  @@map("interests")
}

model UserInterest {
  userId     String   @map("user_id")
  interestId Int      @map("interest_id")
  user       User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  interest   Interest @relation(fields: [interestId], references: [id])
  @@id([userId, interestId])
  @@map("user_interests")
}

model ProjectCategory {
  id       Int       @id @default(autoincrement())
  name     String    @unique
  projects Project[]
  @@map("project_categories")
}

model Project {
  id           String        @id @default(uuid())
  ownerId      String        @map("owner_id")
  categoryId   Int           @map("category_id")
  title        String
  description  String
  difficulty   Level
  status       ProjectStatus @default(DRAFT)
  maxTeamSize  Int           @map("max_team_size")   // includes the owner
  hoursPerWeek Int?          @map("hours_per_week")
  deadline     DateTime?
  githubUrl    String?       @map("github_url")
  demoUrl      String?       @map("demo_url")
  createdAt    DateTime      @default(now()) @map("created_at")
  updatedAt    DateTime      @updatedAt @map("updated_at")
  owner        User                 @relation("ProjectOwner", fields: [ownerId], references: [id])
  category     ProjectCategory      @relation(fields: [categoryId], references: [id])
  skills       ProjectSkill[]
  applications ProjectApplication[]
  members      ProjectMember[]
  @@index([status, categoryId])
  @@index([ownerId])
  @@map("projects")
}

model ProjectSkill {
  projectId String  @map("project_id")
  skillId   Int     @map("skill_id")
  minLevel  Level   @default(BEGINNER) @map("min_level")
  project   Project @relation(fields: [projectId], references: [id], onDelete: Cascade)
  skill     Skill   @relation(fields: [skillId], references: [id])
```

```
  @@id([projectId, skillId])
  @@index([skillId])
  @@map("project_skills")
}

model ProjectApplication {
  id          String            @id @default(uuid())
  projectId   String            @map("project_id")
  applicantId String            @map("applicant_id")
  message     String
  status      ApplicationStatus @default(PENDING)
  createdAt   DateTime          @default(now()) @map("created_at")
  decidedAt   DateTime?         @map("decided_at")
  project       Project        @relation(fields: [projectId], references: [id], onDelete: Cascade)
  applicant     User           @relation(fields: [applicantId], references: [id])
  notifications Notification[]
  @@unique([projectId, applicantId])
  @@index([projectId, status])
  @@index([applicantId])
  @@map("project_applications")
}

model ProjectMember {
  projectId String   @map("project_id")
  userId    String   @map("user_id")
  joinedAt  DateTime @default(now()) @map("joined_at")
  project   Project  @relation(fields: [projectId], references: [id], onDelete: Cascade)
  user      User     @relation(fields: [userId], references: [id])
  @@id([projectId, userId])
  @@index([userId])
  @@map("project_members")
}

model SkillExchange {
  id               String         @id @default(uuid())
  senderId         String         @map("sender_id")
  recipientId      String         @map("recipient_id")
  offeredSkillId   Int            @map("offered_skill_id")
  requestedSkillId Int            @map("requested_skill_id")
  message          String
  status           ExchangeStatus @default(PENDING)
  createdAt        DateTime       @default(now()) @map("created_at")
  updatedAt        DateTime       @updatedAt @map("updated_at")
  sender         User           @relation("ExchangeSender", fields: [senderId], references: [id])
  recipient      User           @relation("ExchangeRecipient", fields: [recipientId], references: [id])
  offeredSkill   Skill          @relation("OfferedSkill", fields: [offeredSkillId], references: [id])
  requestedSkill Skill          @relation("RequestedSkill", fields: [requestedSkillId], references: [id])
  notifications  Notification[]
  @@index([recipientId, status])
  @@index([senderId, status])
  @@map("skill_exchanges")
}

model Notification {
  id            String           @id @default(uuid())
  recipientId   String           @map("recipient_id")
  type          NotificationType
  applicationId String?          @map("application_id")
  exchangeId    String?          @map("exchange_id")
  isRead        Boolean          @default(false) @map("is_read")
  createdAt     DateTime         @default(now()) @map("created_at")
  recipient   User                @relation(fields: [recipientId], references: [id], onDelete: Cascade)
  application ProjectApplication? @relation(fields: [applicationId], references: [id], onDelete: Cascade)
  exchange    SkillExchange?      @relation(fields: [exchangeId], references: [id], onDelete: Cascade)
  @@index([recipientId, isRead, createdAt])
  @@map("notifications")
}
```

# Appendix B. Raw SQL added to the first migration

Prisma cannot express CHECK constraints or partial indexes, so they are appended to the generated migration (`prisma migrate dev --create-only`, then edit the SQL file before applying).

```
ALTER TABLE projects
  ADD CONSTRAINT chk_team_size CHECK (max_team_size BETWEEN 2 AND 10);

ALTER TABLE skill_exchanges
  ADD CONSTRAINT chk_not_self CHECK (sender_id <> recipient_id);

-- Only one active exchange for the same pair and skills
CREATE UNIQUE INDEX uq_active_exchange
  ON skill_exchanges (sender_id, recipient_id, offered_skill_id, requested_skill_id)
  WHERE status IN ('PENDING', 'ACCEPTED');

-- A notification points to exactly one of application or exchange
ALTER TABLE notifications
  ADD CONSTRAINT chk_one_target CHECK (num_nonnulls(application_id, exchange_id) = 1);
```

# Appendix C. Standard error codes

| Code | HTTP | Meaning |
|---|---|---|
| VALIDATION_ERROR | 400 | Request body, query or params failed validation; details lists field errors |
| BAD_REQUEST | 400 | Syntactically valid but not allowed (for example applying to own project) |
| UNAUTHORIZED | 401 | Not logged in, session expired, or wrong credentials |
| FORBIDDEN | 403 | Logged in but lacks permission |
| NOT_FOUND | 404 | Resource missing or hidden from the caller |
| CONFLICT | 409 | Business conflict: full team, already processed, illegal transition, duplicate active exchange |
| DUPLICATE | 409 | Unique constraint violated (for example duplicate email) |
| RATE_LIMITED | 429 | Too many requests |
| INTERNAL_ERROR | 500 | Unexpected failure; details only in server logs |